

Table des matières

Introduction générale	1
1 Organisme d'accueil et système d'information	2
Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.1.1 Présentation de Toulouse INP-ENSEEIH	3
1.1.2 Le service des relations internationales	3
1.2 Analyse du système d'information interne	4
1.2.1 Les outils en place	4
1.2.1.1 QS MoveON	4
1.2.1.2 Pégase	4
1.2.1.3 Eudonet	5
1.2.1.4 Les fichiers Excel	5
1.2.2 Limites structurelles	5
Conclusion	6
2 Analyse des solutions existantes et positionnement	7
Introduction	8
2.1 Grille d'évaluation	8
2.2 Analyse des solutions existantes	8
2.2.1 QS MoveON	9
2.2.2 ErasmusJET	9
2.2.3 Travaux académiques : Elahi et al. (2022)	10
2.3 Tableau comparatif de synthèse	11
2.4 Problématique et solution proposée	11
2.4.1 Problématique	11
2.4.2 Solution proposée	12
Conclusion	12
3 Besoins du système et choix technologiques	13
Introduction	17
3.1 Besoins fonctionnels	17
3.1.1 Fonctionnalités opérationnelles	17
3.1.2 Fonctionnalités intelligentes	17
3.2 Besoins non fonctionnels	18
3.3 Profils utilisateurs et périmètre de réalisation	18

3.3.1	Profils utilisateurs	18
3.3.2	Périmètre de réalisation	19
3.4	Cadre méthodologique et approches de développement	19
3.4.1	Méthodologie de gestion de projet	19
3.4.1.1	Méthode en cascade	20
3.4.1.2	Kanban	20
3.4.1.3	Scrum	20
3.4.1.4	Comparaison et choix retenu	21
3.4.2	Organisation du code : Domain-Driven Design allégé	21
3.4.2.1	Découpage en domaines métier	21
3.4.2.2	Langage ubiquitaire	22
3.4.2.3	Encapsulation des règles métier	22
3.4.3	Style architectural : architecture en couches	22
3.5	Choix du frontend	23
3.5.1	Besoins spécifiques	23
3.5.2	Alternatives évaluées	23
3.5.2.1	Applications monopage — React + Vite	23
3.5.2.2	Angular	24
3.5.2.3	Next.js 14	24
3.5.3	Synthèse comparative	25
3.5.4	Décision	25
3.6	Choix du backend	25
3.6.1	Besoins spécifiques	25
3.6.2	Choix du framework backend	26
3.6.2.1	Alternatives évaluées	26
3.6.2.1.1	Node.js	26
3.6.2.1.2	FastAPI	26
3.6.2.1.3	Django	27
3.6.2.2	Synthèse comparative	27
3.6.2.3	Décision	28
3.6.3	Choix de la couche API	28
3.6.3.1	Besoins spécifiques	28
3.6.3.2	Alternatives évaluées	28
3.6.3.2.1	Django REST Framework (DRF)	28
3.6.3.2.2	Django Ninja	29
3.6.3.3	Synthèse comparative	29
3.6.3.4	Décision	30
3.7	Choix de la base de données	30
3.7.1	Besoins spécifiques	30
3.7.2	Alternatives évaluées	30

3.7.2.1	MySQL	30
3.7.2.2	MongoDB	31
3.7.2.3	PostgreSQL 16	31
3.7.3	Synthèse comparative	32
3.7.4	Décision	32
3.8	Choix du format de stockage des imports bruts	32
3.8.1	Besoins spécifiques	32
3.8.2	Alternatives évaluées	33
3.8.2.1	Fichiers plats (JSON sur le système de fichiers)	33
3.8.2.2	Base NoSQL dédiée (MongoDB)	33
3.8.2.3	Type JSONB dans PostgreSQL	33
3.8.3	Synthèse comparative	34
3.8.4	Décision	34
3.9	Choix du gestionnaire de tâches asynchrones	34
3.9.1	Besoins spécifiques	34
3.9.2	Alternatives évaluées	35
3.9.2.1	Celery + Redis	35
3.9.2.2	Django-Q2 + PostgreSQL	35
3.9.3	Synthèse comparative	36
3.9.4	Décision	36
3.10	Choix du stockage de fichiers	36
3.10.1	Besoins spécifiques	36
3.10.2	Alternatives évaluées	37
3.10.2.1	Système de fichiers local	37
3.10.2.2	MinIO (stockage objet S3-compatible)	37
3.10.3	Synthèse comparative	37
3.10.4	Décision	38
3.11	Couche de routage des requêtes	38
3.11.1	Besoins spécifiques	38
3.11.2	Alternatives évaluées	38
3.11.2.1	Reverse proxy	39
3.11.2.2	API Gateway	39
3.11.3	Synthèse comparative	39
3.11.4	Décision	40
3.11.5	Choix de l'implémentation : Nginx vs Traefik	40
3.11.5.1	Besoins spécifiques	40
3.11.5.2	Alternatives évaluées	40
3.11.5.2.1	Nginx	40
3.11.5.2.2	Traefik	40
3.11.5.3	Synthèse comparative	41

3.11.5.4	Décision	41
3.12	Stratégie de performance base de données	41
3.12.1	Contexte et enjeux	41
3.12.2	Alternatives évaluées	42
3.12.2.1	Partitionnement par année	42
3.12.2.2	Indexation ciblée	42
3.12.3	Synthèse comparative	42
3.12.4	Décision	43
3.13	Choix de l'algorithme d'affectation	43
3.13.1	Besoins spécifiques	43
3.13.2	Alternatives évaluées	43
3.13.2.1	Algorithme glouton	44
3.13.2.2	Gale-Shapley	44
3.13.3	Synthèse comparative	44
3.13.4	Décision	45
3.14	Choix du moteur de recommandation	45
3.14.1	Besoins spécifiques	45
3.14.2	Alternatives évaluées	45
3.14.2.1	Approche basée sur des règles (rule-based)	46
3.14.2.2	Approche par similarité (collaborative filtering)	46
3.14.2.3	Approche par apprentissage supervisé	46
3.14.2.3.1	Régression logistique	46
3.14.2.3.2	XGBoost	47
3.14.3	Synthèse comparative	47
3.14.4	Décision	47
3.15	Choix de l'orchestration RAG	48
3.15.1	Besoins spécifiques	48
3.15.2	Modèle de langage	48
3.15.3	Alternatives évaluées	49
3.15.3.1	LangChain	49
3.15.3.2	LlamaIndex	49
3.15.3.3	Haystack	49
3.15.3.4	Approche directe (sans framework)	50
3.15.4	Synthèse comparative	50
3.15.5	Décision	50
3.16	Synthèse des choix technologiques	51
	Conclusion	52

4	Conception du système	53
	Introduction	55
4.1	Product Backlog	55
4.1.1	Priorités MoSCoW	55
4.1.2	Story points	55
4.1.3	Tableau du Product Backlog	56
4.2	Planification des sprints	59
4.2.1	Diagramme de Gantt	60
4.3	Architecture logicielle	63
4.3.1	Vue d'ensemble	63
4.3.2	Couche présentation — Next.js 14	64
4.3.3	Couche applicatif — Django 5 + Django Ninja	64
4.3.4	Couche traitement asynchrone — Django-Q2	64
4.3.5	Couche données — PostgreSQL 16	64
4.3.6	Couche intégration — Adaptateurs ETL	65
4.3.6.1	Stratégie d'upsert	65
4.3.6.2	Clé de jointure : l'INE	66
4.3.6.3	Stratégie de synchronisation	66
4.3.7	Sécurité et contrôle d'accès	66
4.3.7.1	Flux d'authentification OIDC	66
4.3.7.2	Contrôle d'accès (RBAC)	66
4.3.7.3	Gestion du renouvellement des tokens JWT	67
4.3.7.4	Journal d'audit	67
4.3.7.5	Protection des données (RGPD)	68
4.4	Architecture physique et déploiement	68
4.4.1	Vue d'ensemble	68
4.4.2	Containers Docker	69
4.4.3	Flux réseau	69
4.4.4	Environnements	70
4.4.5	CI/CD	70
4.4.6	Stratégie de migration de schéma	70
4.4.7	Stratégie de rollback	70
4.4.8	Politique de sauvegarde	71
4.4.9	Gestion des secrets	71
4.5	Modèle de données	72
4.5.1	Organisation par packages	72
4.5.2	Principes structurants	72
4.5.2.1	L'année universitaire comme axe temporel.	72
4.5.2.2	L'INE comme clé de jointure universelle.	73
4.5.2.3	Séparation quotas INP / quotas N7.	73

4.6	Machines à états finis (FSM)	73
4.6.1	Principe et implémentation	73
4.6.2	FSM de l'année universitaire	73
4.6.3	FSM des entités métier	74
4.7	Flux de données : de la source au workflow	75
4.7.1	Détail de l'étape de nettoyage et validation	75
4.7.1.1	Description du processus de transformation	76
4.7.1.1.1	Gestion des sources multiples	76
4.7.1.1.2	Standardisation des formats	76
4.7.1.1.3	Normalisation sémantique	76
4.7.1.1.4	Gestion des valeurs manquantes	77
4.7.1.1.5	Détection et gestion des doublons	77
4.7.1.1.6	Validation métier	77
4.7.1.1.7	Traçabilité et gestion des erreurs	78
4.7.1.2	Architecture du pipeline de nettoyage	78
4.7.1.3	Technologies et outils utilisés	79
4.8	Gestion des risques	79
	Conclusion	80
5	Réalisation	81
	Introduction	82
5.1	Sprint 1 — Socle technique et référentiels	82
5.1.1	Objectif du sprint	82
5.1.2	Sprint Backlog — Sprint 1	82
5.1.3	Conception détaillée	84
5.1.3.1	Diagramme de classes	84
5.1.4	Implémentation	85
5.1.5	Tests	85
5.2	Sprint 2 — Données et traçabilité	85
5.2.1	Objectif du sprint	85
5.2.2	Sprint Backlog — Sprint 2	86
5.2.3	Conception détaillée	88
5.2.3.1	Diagramme de classes	88
5.2.4	Implémentation	89
5.2.5	Tests	89
	Conclusion générale	90
	Bibliographie	91

Table des figures

4.1	Architecture logicielle	63
4.2	Architecture physique	68
4.3	Machine à états — AcademicYear	73
4.4	Pipeline global de traitement des données	75
4.5	Pipeline détaillé de l'étape de nettoyage des données	78
5.1	Diagramme de classes — Sprint 1	85
5.2	Diagramme de classes — Sprint 2	89

Liste des tableaux

2.1	Grille d'évaluation des solutions existantes	8
2.2	Synthèse comparative des solutions selon la grille d'évaluation	11
3.1	Comparaison des approches de gestion de projet	21
3.2	Découpage en domaines métier du système	22
3.3	Comparaison des frameworks frontend	25
3.4	Comparaison des frameworks backend	28
3.5	Comparaison DRF et Django Ninja	30
3.6	Comparaison des systèmes de gestion de base de données	32
3.7	Comparaison des formats de stockage pour <code>RawImport</code>	34
3.8	Comparaison des gestionnaires de tâches asynchrones	36
3.9	Comparaison des solutions de stockage de fichiers	38
3.10	Comparaison des approches de routage des requêtes	39
3.11	Comparaison des outils de reverse proxy	41
3.12	Comparaison des stratégies de performance base de données	43
3.13	Comparaison des algorithmes d'affectation	45
3.14	Comparaison des approches de recommandation	47
3.15	Variables d'entrée du moteur de recommandation	48
3.16	Comparaison des frameworks d'orchestration RAG	50
3.17	Synthèse des choix technologiques	51
4.1	Échelle des story points	55
4.2	Product Backlog	56
4.3	Planification des sprints	59
4.4	Diagramme de Gantt	61
4.5	Stratégie de synchronisation par source	66
4.6	Containers Docker et leurs rôles	69
4.7	Packages du modèle de données	72
4.8	Conditions préalables (guard conditions) des transitions FSM	74
4.9	FSM des entités métier	74
4.10	Matrice des risques	79

Liste des abréviations

CAS	Central Authentication Service
CTI	Commission des Titres d'Ingénieur
GPA	Grade Point Average
INP	Institut National Polytechnique
RI	Relations Internationales

Introduction générale

Organisme d'accueil et système d'information

Plan

Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.1.1 Présentation de Toulouse INP-ENSEEIH	3
1.1.2 Le service des relations internationales	3
1.2 Analyse du système d'information interne	4
1.2.1 Les outils en place	4
1.2.1.1 QS MoveON	4
1.2.1.2 Pégase	4
1.2.1.3 Eudonet	5
1.2.1.4 Les fichiers Excel	5
1.2.2 Limites structurelles	5
Conclusion	6

Introduction

Ce chapitre présente l’organisme d’accueil, le service des Relations Internationales de l’ENSEEIH, et décrit les outils actuellement utilisés pour la gestion des mobilités, ainsi que les limites structurelles qui ont motivé ce projet.

1.1 Présentation de l’organisme d’accueil

Ce stage a été effectué au sein de Toulouse INP-ENSEEIH, école d’ingénieurs publique toulousaine, et plus précisément dans son service des Relations Internationales. Cette partie présente d’abord l’établissement d’accueil, puis le service dans lequel j’ai effectué mon stage.

1.1.1 Présentation de Toulouse INP-ENSEEIH

Toulouse INP-ENSEEIH est une grande école publique d’ingénieurs, située au cœur de Toulouse et composante de l’Institut national polytechnique de Toulouse. Accréditée à délivrer le diplôme d’ingénieur, elle forme chaque année plusieurs centaines d’ingénieurs et des étudiants en master et en doctorat, dans des domaines : Sciences du Numérique (SN), Électronique–Énergie électrique–Automatique (EEEA) et Mécanique des Fluides, Énergétique & Environnement (MFEE).

L’école maintient des liens étroits avec plusieurs laboratoires de recherche de référence, tels que IMFT, IRIT, LAAS et LAPLACE, et des relations solides avec de nombreux partenaires industriels. L’international occupe également une place centrale dans sa stratégie de développement : l’école élargit continuellement son réseau d’universités partenaires et encourage activement les expériences de mobilité à l’étranger pour ses étudiants [1].

1.1.2 Le service des relations internationales

Le service des Relations Internationales (RI) de l’ENSEEIH pilote l’ensemble de la politique d’ouverture internationale de l’école. Il joue un rôle central à la fois dans le développement des partenariats académiques et dans le suivi opérationnel des mobilités étudiantes. Ses missions principales couvrent six grands axes [2].

La gestion des partenariats académiques : administration d’un réseau d’accords bilatéraux avec des universités étrangères, couvrant les programmes d’échange et les doubles diplômes, avec suivi des quotas de places par accord et par département.

L’organisation des mobilités sortantes : pilotage de la campagne annuelle de sélection et d’affectation des étudiants de l’ENSEEIH partant à l’étranger, de la publication des destinations jusqu’à la validation des résultats [3].

L’accueil des mobilités entrantes : enregistrement, rattachement administratif et suivi des étudiants internationaux accueillis dans le cadre des accords de partenariat [4].

L'accompagnement administratif et pédagogique : information des étudiants sur les procédures, gestion des aspects financiers, et vérification de la conformité des dossiers avec les exigences académiques.

Le pilotage et le reporting CTI : suivi des flux entrants et sortants pour les audits de la Commission des Titres d'Ingénieur, qui impose une durée minimale de mobilité internationale à chaque étudiant pour l'obtention du diplôme d'ingénieur.

La gestion des programmes courts : suivi des étudiants participant à des summer schools, séjours linguistiques ou autres programmes de courte durée à l'étranger, qui peuvent être pris en compte dans le calcul de la durée de mobilité sous certaines conditions [5].

Ces missions impliquent de manipuler quotidiennement des volumes importants de données issues de sources multiples et hétérogènes, ce qui constitue l'un des enjeux centraux du projet décrit dans ce rapport.

1.2 Analyse du système d'information interne

Avant de concevoir une solution adaptée, il est nécessaire de réaliser une analyse des outils et des méthodes utilisés actuellement au sein du service des Relations Internationales. Cette section présente tout d'abord les différents systèmes institutionnels mis en place puis met en lumière les principales limites et difficultés de ce mode de fonctionnement.

1.2.1 Les outils en place

La gestion des mobilités repose sur trois systèmes institutionnels qui ne communiquent pas entre eux, complétés par des fichiers Excel locaux.

1.2.1.1 QS MoveON

MoveON est la plateforme centrale du service RI. Elle gère l'ensemble des accords bilatéraux, les quotas globaux INP et les candidatures post-sélection. Elle expose une API REST permettant d'en extraire les données. C'est l'outil le plus riche de l'écosystème actuel [6], mais son périmètre reste limité : il ne tient pas compte des résultats académiques des étudiants, ne propose pas de mécanisme d'aide à la sélection, et ne prend pas en compte les stages ou autres éléments du parcours individuel.

1.2.1.2 Pégase

Pégase est le système d'information académique de Toulouse INP. Il rassemble les informations d'inscription et les notes — notamment le GPA trimestriel, élément essentiel pour le classement des candidatures. Les données sont accessibles via API, mais aucun lien automatique n'existe avec MoveON.

1.2.1.3 Eudonet

Eudonet est le CRM utilisé pour la gestion des conventions de stage. Il centralise les informations relatives aux stages nationaux et internationaux (entreprise d'accueil, dates, durée) [7]. Ces données sont importantes pour le calcul de la durée de mobilité CTI, mais elles ne sont pas intégrées automatiquement dans les autres systèmes .

1.2.1.4 Les fichiers Excel

Pour compenser les lacunes des trois systèmes précédents, le service RI maintient une collection de fichiers Excel dédiés au suivi des quotas par département, à la gestion des mobilités entrantes, au calcul des durées et au suivi des mobilités complémentaires. Cette approche fonctionne à court terme, mais reste fragile : une simple erreur de manipulation, une mauvaise copie ou un oubli de mise à jour suffit à créer des incohérences dans les données.

1.2.2 Limites structurelles

L'analyse du fonctionnement actuel fait apparaître plusieurs problèmes structurels.

- **Aucune intégration entre les systèmes.** Les outils ne communiquent pas entre eux, ce qui oblige les équipes à multiplier les exports manuels depuis différentes plateformes avant de retraiter les données dans Excel. Ces manipulations sont chronophages, répétitives et génèrent un risque d'erreurs difficile à maîtriser.
- **Absence d'aide à la décision pour l'affectation.** La sélection se fait manuellement, en comparant les GPA dans un tableau. Sans algorithme, les égalités sont tranchées à l'appréciation, et la cohérence des décisions d'une année sur l'autre n'est pas garantie.
- **Manque de granularité des quotas.** MoveON gère les quotas à l'échelle de l'INP global. La répartition par école (ENSEEIH) puis par département constitue une couche supplémentaire maintenue dans Excel, en dehors de tout système structuré.
- **Absence de traçabilité des mobilités complémentaires.** Les summer schools, séjours linguistiques et programmes courts ne font l'objet d'aucun suivi structuré, ce qui fragilise leur prise en compte dans le calcul de la durée de mobilité.
- **Calcul de la durée de mobilité non automatisable.** En raison de la dispersion des données entre plusieurs systèmes, agréger fiablement toutes les périodes de mobilité d'un étudiant est un exercice manuel long et sujet à erreurs.

Conclusion

L'analyse de l'existant met en évidence un environnement fragmenté, où le manque de communication entre outils oblige encore à des traitements manuels sous Excel, peu fiables. Cela met en évidence le besoin d'une solution plus intégrée, présenté dans la suite du rapport.

Analyse des solutions existantes et positionnement

Plan

Introduction	8
2.1 Grille d'évaluation	8
2.2 Analyse des solutions existantes	8
2.2.1 QS MoveON	9
2.2.2 ErasmusJET	9
2.2.3 Travaux académiques : Elahi et al. (2022)	10
2.3 Tableau comparatif de synthèse	11
2.4 Problématique et solution proposée	11
2.4.1 Problématique	11
2.4.2 Solution proposée	12
Conclusion	12

Introduction

Ce chapitre présente les solutions existantes afin d'évaluer leur capacité à répondre aux besoins identifiés. Il permet de déterminer leur pertinence dans le contexte du service RI avant d'envisager une nouvelle solution.

2.1 Grille d'évaluation

Pour comparer les solutions de façon rigoureuse, une grille de huit critères a été définie à partir des problèmes concrets identifiés au chapitre précédent. Chaque critère traduit une lacune précise du système actuel.

#	Critère	Ce que cela signifie concrètement pour l'ENSEEIH
C1	Centralisation des données de mobilité	Regrouper dans une seule plateforme l'ensemble des données relatives aux mobilités sortantes, entrantes, complémentaires et aux stages internationaux, ainsi que les données académiques des étudiants
C2	Quotas par école et par département	Gestion des places allouées d'abord à l'échelle de l'ENSEEIH, puis par département pédagogique (SN, 3EA, MF2E), et non uniquement à l'échelle globale INP
C3	Aide à la décision et auto-affectation	Présence d'un mécanisme automatique de classement et de pré-affectation des candidats.
C4	Calcul de la durée de mobilité	Agrégation automatique de toutes les périodes de mobilité d'un étudiant (académique, stage, complémentaire) pour produire la durée réglementaire
C5	Mobilités complémentaires	Prise en charge des expériences hors accord bilatéral (summer schools, séjours linguistiques, programmes courts) avec dépôt et validation de justificatifs
C6	Adaptabilité aux règles internes	Possibilité de configurer des règles propres à l'établissement — priorités d'affectation, seuils, critères locaux — sans modifier le code source
C7	Recommandation de destinations	Suggestion de destinations adaptées au profil de l'étudiant (département, GPA, langue, préférences) pour l'accompagner dans son choix
C8	Reporting CTI	Production automatique des indicateurs clés (KPIs) exigés par la CTI : taux de mobilité par promotion, durées moyennes, répartition géographique

TABLE 2.1 – Grille d'évaluation des solutions existantes

2.2 Analyse des solutions existantes

Trois solutions ont été examinées au regard de la grille d'évaluation définie précédemment : la plateforme QS MoveON, déjà en place au sein du service RI, la solution Erasmus-JET développée dans le cadre du programme Erasmus+, et les travaux académiques d'Elahi et al. portant sur la recommandation de destinations.

2.2.1 QS MoveON

MoveON est la plateforme commerciale de gestion de la mobilité la plus répandue dans les établissements d'enseignement supérieur français. Sa position dans l'écosystème de l'EN-SEEIHT est particulière : c'est déjà l'outil principal du service RI, ce qui en fait la première solution à examiner — non pas comme un système à remplacer, mais pour cerner précisément ce qu'il ne couvre pas [6].

Points forts :

MoveON gère efficacement le référentiel des accords bilatéraux, les candidatures post-sélection et le suivi administratif des dossiers étudiants. Son API REST permet d'extraire les données.

Limites par rapport aux critères définis :

- MoveON ne couvre qu'une partie des données : les mobilités sortantes et les accords bilatéraux. Les données académiques, les stages internationaux et les mobilités complémentaires sont absents. Le service RI est contraint d'exporter les données de MoveON, de les compléter manuellement dans Excel, puis de travailler sur ce fichier hybride — c'est précisément cette friction que le projet cherche à éliminer.
- MoveON gère les quotas à l'échelle de l'INP global. La granularité d'abord par école (ENSEEIHT), puis par département pédagogique, qui conditionne pourtant les règles d'affectation internes, est entièrement absente et maintenue dans des fichiers Excel en dehors de tout système structuré.
- Aucun mécanisme de classement ou d'affectation automatique n'existe. La sélection se fait entièrement à la main.
- MoveON ne connaît ni les stages, ni les expériences complémentaires. Le calcul de la durée de mobilité est donc impossible à automatiser depuis MoveON seul.
- MoveON ne propose aucun mécanisme de suggestion de destinations adapté au profil de l'étudiant.
- MoveON ne produit pas les indicateurs CTI structurés (taux de mobilité par promotion, durées moyennes, répartition géographique) dans un format exploitable directement pour les audits.

MoveON reste un bon outil pour ce qu'il fait, mais sa conception ne prévoit pas de couvrir les besoins opérationnels internes d'un établissement.

2.2.2 ErasmusJET

ErasmusJET est une solution de gestion de la mobilité développée dans le cadre des initiatives de digitalisation du programme Erasmus+. Elle vise à centraliser la gestion des mobilités entrantes et sortantes au sein du réseau Erasmus, en standardisant les échanges de données entre partenaires (candidatures, conventions, transcripts).

Points forts :

ErasmusJET simplifie les processus administratifs liés aux mobilités Erasmus+ : dépôt de candidatures en ligne, génération et signature numérique des conventions, échange automatique de relevés de notes entre institutions. Pour un établissement très fortement orienté Erasmus, c’est un outil cohérent.

Limites par rapport aux critères définis :

- ErasmusJET se limite aux mobilités Erasmus+. Les données académiques, les stages internationaux et les mobilités hors programme européen restent entièrement en dehors de son périmètre.
- Les quotas par école et par département, l’auto-affectation, le calcul de la durée de mobilité et les expériences complémentaires hors Erasmus+ sont entièrement hors de son périmètre. Pour l’ENSEEIH, une part importante des mobilités repose sur des accords bilatéraux hors Europe, ce qui réduit encore davantage la couverture réelle d’ErasmusJET.
- La plateforme est standardisée à l’échelle européenne. Les règles métier propres à l’établissement ne peuvent pas être configurées.
- Aucune recommandation de destinations ni production d’indicateurs CTI structurés ne sont prévues.

ErasmusJET peut compléter MoveON pour le volet Erasmus+, mais ne répond à aucun des problèmes structurels identifiés.

2.2.3 Travaux académiques : Elahi et al. (2022)

Elahi et al. proposent une architecture de système de gestion de mobilité étudiante intégrant un moteur de recommandation d’universités basé sur le filtrage collaboratif et les préférences des étudiants. Les auteurs modélisent le processus d’affectation comme un problème de mise en correspondance entre étudiants et destinations, et évaluent leur approche sur un jeu de données simulé représentant les vœux et profils académiques d’étudiants. [8]

Points forts :

L’article démontre la faisabilité technique de coupler un algorithme de recommandation avec un processus de sélection. Sur le critère C7, c’est la seule solution analysée qui propose une approche structurée de recommandation de destinations adaptée au profil de l’étudiant.

Limites par rapport aux critères définis :

- Le modèle ne propose aucune centralisation des données de mobilité. Il repose sur un dataset uniforme et simulé, sans connexion avec des systèmes institutionnels réels.
- Les quotas par école et par département, l’auto-affectation sous contraintes, le calcul de la durée de mobilité et la gestion des mobilités complémentaires sont absents du modèle, qui se concentre uniquement sur la recommandation.

- Étant un prototype de recherche, le système ne prévoit aucun mécanisme de configuration des règles métier propres à un établissement.
- La dimension réglementaire du reporting, centrale pour un établissement accrédité CTI, est entièrement absente du modèle.

Ces travaux constituent une référence utile pour la partie recommandation du projet, mais ils ne proposent pas de solution opérationnelle transposable à l'ENSEEIH.

2.3 Tableau comparatif de synthèse

Le tableau suivant résume la couverture de chaque solution pour les huit critères de la grille d'évaluation.

Critère	QS MoveON	ErasmusJET	Elahi et al.
C1 – Centralisation des données	Partiel	✗	✗
C2 – Quotas par école et département	✗	✗	✗
C3 – Aide à la décision et auto-affectation	✗	✗	✗
C4 – Calcul durée de mobilité	✗	✗	✗
C5 – Mobilités complémentaires	✗	✗	✗
C6 – Adaptabilité métier	Limitée	✗	✗
C7 – Recommandation de destinations	✗	✗	Partiel
C8 – Reporting CTI	Partiel	✗	✗

TABLE 2.2 – Synthèse comparative des solutions selon la grille d'évaluation

Aucune des solutions analysées ne satisfait l'ensemble des critères définis. Ce constat justifie la conception d'une solution spécifique.

2.4 Problématique et solution proposée

Cette section formule la problématique centrale du projet, puis présente la solution retenue.

2.4.1 Problématique

Comment concevoir une plateforme intégrée qui automatise le processus de gestion des mobilités internationales à l'ENSEEIH, depuis la centralisation des données multi-sources jusqu'à l'affectation algorithmique, tout en garantissant la conformité CTI et l'aide à la décision des étudiants ?

2.4.2 Solution proposée

La réponse ne passe pas par le remplacement de MoveON ou des autres outils en place, mais par le développement d'une couche d'intégration et d'automatisation qui vient se greffer sur l'existant, en répondant à l'ensemble des huit critères définis dans la grille d'évaluation.

Conclusion

L'étude des solutions existantes montre qu'aucun des outils analysés ne répond pleinement aux besoins du service RI. Ce constat conduit donc à définir plus précisément les besoins du système ainsi que les choix technologiques les plus adaptés.

Besoins du système et choix technologiques

Plan

Introduction	17
3.1 Besoins fonctionnels	17
3.1.1 Fonctionnalités opérationnelles	17
3.1.2 Fonctionnalités intelligentes	17
3.2 Besoins non fonctionnels	18
3.3 Profils utilisateurs et périmètre de réalisation	18
3.3.1 Profils utilisateurs	18
3.3.2 Périmètre de réalisation	19
3.4 Cadre méthodologique et approches de développement	19
3.4.1 Méthodologie de gestion de projet	19
3.4.1.1 Méthode en cascade	20
3.4.1.2 Kanban	20
3.4.1.3 Scrum	20
3.4.1.4 Comparaison et choix retenu	21
3.4.2 Organisation du code : Domain-Driven Design allégé	21
3.4.2.1 Découpage en domaines métier	21
3.4.2.2 Langage ubiquitaire	22
3.4.2.3 Encapsulation des règles métier	22
3.4.3 Style architectural : architecture en couches	22
3.5 Choix du frontend	23
3.5.1 Besoins spécifiques	23
3.5.2 Alternatives évaluées	23
3.5.2.1 Applications monopage — React + Vite	23
3.5.2.2 Angular	24
3.5.2.3 Next.js 14	24
3.5.3 Synthèse comparative	25
3.5.4 Décision	25
3.6 Choix du backend	25
3.6.1 Besoins spécifiques	25
3.6.2 Choix du framework backend	26
3.6.2.1 Alternatives évaluées	26
3.6.2.1.1 Node.js	26

3.6.2.1.2	FastAPI	26
3.6.2.1.3	Django	27
3.6.2.2	Synthèse comparative	27
3.6.2.3	Décision	28
3.6.3	Choix de la couche API	28
3.6.3.1	Besoins spécifiques	28
3.6.3.2	Alternatives évaluées	28
3.6.3.2.1	Django REST Framework (DRF)	28
3.6.3.2.2	Django Ninja	29
3.6.3.3	Synthèse comparative	29
3.6.3.4	Décision	30
3.7	Choix de la base de données	30
3.7.1	Besoins spécifiques	30
3.7.2	Alternatives évaluées	30
3.7.2.1	MySQL	30
3.7.2.2	MongoDB	31
3.7.2.3	PostgreSQL 16	31
3.7.3	Synthèse comparative	32
3.7.4	Décision	32
3.8	Choix du format de stockage des imports bruts	32
3.8.1	Besoins spécifiques	32
3.8.2	Alternatives évaluées	33
3.8.2.1	Fichiers plats (JSON sur le système de fichiers)	33
3.8.2.2	Base NoSQL dédiée (MongoDB)	33
3.8.2.3	Type JSONB dans PostgreSQL	33
3.8.3	Synthèse comparative	34
3.8.4	Décision	34
3.9	Choix du gestionnaire de tâches asynchrones	34
3.9.1	Besoins spécifiques	34
3.9.2	Alternatives évaluées	35
3.9.2.1	Celery + Redis	35
3.9.2.2	Django-Q2 + PostgreSQL	35
3.9.3	Synthèse comparative	36
3.9.4	Décision	36
3.10	Choix du stockage de fichiers	36
3.10.1	Besoins spécifiques	36
3.10.2	Alternatives évaluées	37
3.10.2.1	Système de fichiers local	37
3.10.2.2	MinIO (stockage objet S3-compatible)	37
3.10.3	Synthèse comparative	37

3.10.4	Décision	38
3.11	Couche de routage des requêtes	38
3.11.1	Besoins spécifiques	38
3.11.2	Alternatives évaluées	38
3.11.2.1	Reverse proxy	39
3.11.2.2	API Gateway	39
3.11.3	Synthèse comparative	39
3.11.4	Décision	40
3.11.5	Choix de l'implémentation : Nginx vs Traefik	40
3.11.5.1	Besoins spécifiques	40
3.11.5.2	Alternatives évaluées	40
3.11.5.2.1	Nginx	40
3.11.5.2.2	Traefik	40
3.11.5.3	Synthèse comparative	41
3.11.5.4	Décision	41
3.12	Stratégie de performance base de données	41
3.12.1	Contexte et enjeux	41
3.12.2	Alternatives évaluées	42
3.12.2.1	Partitionnement par année	42
3.12.2.2	Indexation ciblée	42
3.12.3	Synthèse comparative	42
3.12.4	Décision	43
3.13	Choix de l'algorithme d'affectation	43
3.13.1	Besoins spécifiques	43
3.13.2	Alternatives évaluées	43
3.13.2.1	Algorithme glouton	44
3.13.2.2	Gale-Shapley	44
3.13.3	Synthèse comparative	44
3.13.4	Décision	45
3.14	Choix du moteur de recommandation	45
3.14.1	Besoins spécifiques	45
3.14.2	Alternatives évaluées	45
3.14.2.1	Approche basée sur des règles (rule-based)	46
3.14.2.2	Approche par similarité (collaborative filtering)	46
3.14.2.3	Approche par apprentissage supervisé	46
3.14.2.3.1	Régression logistique	46
3.14.2.3.2	XGBoost	47
3.14.3	Synthèse comparative	47
3.14.4	Décision	47
3.15	Choix de l'orchestration RAG	48

3.15.1	Besoins spécifiques	48
3.15.2	Modèle de langage	48
3.15.3	Alternatives évaluées	49
3.15.3.1	LangChain	49
3.15.3.2	LlamaIndex	49
3.15.3.3	Haystack	49
3.15.3.4	Approche directe (sans framework)	50
3.15.4	Synthèse comparative	50
3.15.5	Décision	50
3.16	Synthèse des choix technologiques	51
	Conclusion	52

Introduction

Ce chapitre détaille les besoins fonctionnels et non fonctionnels du système, présente les profils utilisateurs et le périmètre de réalisation, puis justifie les choix technologiques retenus pour chaque composant du stack.

3.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les principales fonctionnalités attendues du système afin de répondre aux problématiques identifiées au sein du service RI. Ils couvrent à la fois les fonctionnalités opérationnelles et les fonctionnalités intelligentes

3.1.1 Fonctionnalités opérationnelles

Le système prend en charge les opérations suivantes, qui couvrent l'ensemble du cycle de gestion des mobilités internationales, de l'import des données jusqu'à l'archivage des résultats.

- **Intégration des données** : import depuis MoveON, Pégase et Eudonet via API REST, et depuis des fichiers Excel pour les sources sans API. Chaque import inclut une détection de doublons, une validation des formats et un mécanisme de fallback Excel en cas d'indisponibilité d'une API.
- **Gestion des accords et quotas** : import des accords depuis MoveON et gestion de la répartition locale des places par département N7, avec possibilité d'ajustement manuel par le service RI.
- **Algorithme d'affectation** : pré-affectation automatique basée sur le GPA, les vœux ordonnés et les quotas par département, avec interface de supervision et d'ajustement.
- **Suivi CTI** : calcul automatique de la durée totale de mobilité par agrégation des séjours académiques, stages et expériences complémentaires validées.
- **Mobilités complémentaires** : les étudiants déclarent leurs expériences et déposent les justificatifs nécessaires, puis le service RI se charge de la validation des dossiers.
- **Mobilités entrantes** : import via template Excel et suivi administratif granulaire.
- **Historisation** : les données d'une campagne clôturée passent en lecture seule. Aucune exécution ultérieure ne peut modifier les résultats d'une année passée.

3.1.2 Fonctionnalités intelligentes

Les fonctionnalités intelligentes sont identifiées comme des extensions V2. Elles sont conservées dans l'analyse comme pistes d'évolution, mais ne font pas partie du périmètre MVP engagé.

3.2 Besoins non fonctionnels

Au-delà des fonctionnalités métier, le système doit également répondre à plusieurs exigences techniques et organisationnelles. Ces besoins non fonctionnels concernent notamment la sécurité, la performance, l'intégrité des données et l'intégration avec l'environnement existant.

- **Interopérabilité** : le système doit s'intégrer sans rupture dans l'écosystème existant via des adaptateurs REST dédiés à chaque source. En cas d'indisponibilité d'une API, un fallback par import Excel garantit la continuité opérationnelle.
- **Intégrité des données** : les données provenant de sources hétérogènes peuvent être incohérentes ou dupliquées. Une validation stricte est appliquée à chaque import — contrôle de format, détection de doublons, cohérence inter-sources. Toute donnée rejetée est tracée dans un rapport d'import avec la raison précise, pour correction ciblée.
- **Sécurité et RGPD** : authentification via le SSO institutionnel, gestion stricte des rôles (étudiant vs administrateur), chiffrement des données sensibles. Un étudiant n'accède qu'à son propre dossier.
- **Performance** : le temps de réponse de l'API pour les endpoints courants doit être inférieur à 200 ms sous une charge de 50 utilisateurs simultanés. L'algorithme d'affectation doit s'exécuter en moins de 30 secondes pour une promotion de 300 étudiants. Les traitements lourds — algorithme d'affectation, calculs CTI — sont exécutés de façon asynchrone en arrière-plan pour ne pas bloquer l'interface.
- **Scalabilité** : séparation entre données actives et données archivées en lecture seule pour absorber la croissance annuelle du volume historique et les pics de connexion lors des moments clés de campagne.
- **Ergonomie** : l'interface adapte sa navigation et ses fonctionnalités au rôle de l'utilisateur connecté, réduisant les erreurs de manipulation.
- **Traçabilité** : toute action importante — import, modification d'affectation, validation, transition de phase — est enregistrée avec l'identité de l'auteur et l'horodatage dans un journal d'audit persistant en écriture seule.

3.3 Profils utilisateurs et périmètre de réalisation

Cette section présente les profils d'utilisateurs du système, puis définit le périmètre de réalisation retenu pour ce projet.

3.3.1 Profils utilisateurs

Deux profils utilisent le système, avec des besoins très différents.

L’administrateur — c’est-à-dire le service RI — pilote la campagne de bout en bout. Il supervise les imports, configure les quotas, ajuste les propositions d’affectation, instruit les demandes de mobilités complémentaires et produit les rapports CTI. Le système prend en charge les tâches répétitives, mais toute décision finale reste sous sa responsabilité, et chaque intervention manuelle est tracée.

L’étudiant consulte les destinations disponibles, suit l’état de sa candidature, déclare ses expériences complémentaires et vérifie à tout moment sa conformité CTI. Les recommandations personnalisées et l’assistant virtuel restent des extensions V2 conditionnelles.

3.3.2 Périmètre de réalisation

Le développement est organisé en deux niveaux de priorité.

MVP — fonctionnalités prioritaires :

- Intégration des données multi-sources (MoveON, Pégase, Eudonet, Excel)
- Gestion des accords et quotas par département
- Algorithme d’affectation avec validation humaine
- Calcul automatique de la durée de mobilité CTI
- Gestion des mobilités complémentaires et entrantes
- Tableaux de bord analytiques avancés et exports CTI
- Espace étudiant : catalogue, suivi de candidature, déclarations
- Sécurité : SSO, contrôle d’accès, audit log

V2 — fonctionnalités secondaires :

- Moteur de recommandation de destinations personnalisé
- Assistant virtuel (chatbot RAG)

Ces fonctionnalités V2 sont des *Could Have*. Elles ne seront intégrées à un sprint que si le MVP est réalisé et validé dans les sprints prioritaires, avec une capacité restante suffisante.

3.4 Cadre méthodologique et approches de développement

La conduite du projet repose sur trois niveaux complémentaires : une méthodologie de gestion de projet, une approche d’organisation du code, et un style architectural.

3.4.1 Méthodologie de gestion de projet

Le projet s’étend sur six mois dans un contexte où certaines règles métier n’étaient pas complètement formalisées dès le départ. Trois approches ont été évaluées avant de retenir la méthodologie la mieux adaptée.

3.4.1.1 Méthode en cascade

La méthode en cascade organise le projet en phases séquentielles et strictement ordonnées : analyse, conception, développement, test, déploiement. Chaque phase doit être complètement terminée avant de passer à la suivante.

Avantages : Elle offre une planification claire et une documentation exhaustive dès le départ, ce qui facilite le suivi et la traçabilité des décisions.

Limites dans notre contexte : Elle impose de geler un cahier des charges complet avant tout développement. Or, certaines règles métier — notamment la hiérarchie des quotas et les critères de priorité dans l’affectation — n’étaient pas entièrement définies au démarrage du projet, rendant cette approche inadaptée.

3.4.1.2 Kanban

Kanban est une méthode de gestion de flux continu, sans itérations fixes. Le travail est visualisé sur un tableau et les tâches avancent au fur et à mesure de la disponibilité de l’équipe.

Avantages : Sa flexibilité permet de s’adapter rapidement aux changements et de traiter les tâches par ordre de priorité sans contrainte de cycle.

Limites dans notre contexte : L’absence d’itérations fixes et de jalons structurés le rend peu adapté à un projet avec un périmètre défini et une date de livraison fixe, où une planification rigoureuse est nécessaire.

3.4.1.3 Scrum

Scrum est un cadre agile qui organise le développement en sprints de durée fixe, avec des événements régulières (planning, review, retrospective) et un backlog priorisé.

Avantages : Il offre un bon équilibre entre structure et flexibilité : les priorités peuvent être révisées à chaque sprint, les livraisons sont incrémentales et validables régulièrement par le service RI.

Limites dans notre contexte : Scrum est conçu pour des équipes, ce qui nécessite une adaptation des rôles et des événements pour un projet porté par un seul développeur

3.4.1.4 Comparaison et choix retenu

Critère	Cascade	Kanban	Scrum
Planification	Rigide dès le début	Continue, sans itérations fixes	Sprints fixes avec rétroplanification
Adaptation aux changements	Difficile	Bonne, mais sans rythme structuré	Bonne, intégrée à chaque sprint
Visibilité client	Faible en cours de projet	Modérée	Élevée (Sprint Review)
Adapté à ce projet	✗	Partiel	✓

TABLE 3.1 – Comparaison des approches de gestion de projet

Scrum est retenu pour son équilibre entre structure (sprints de durée fixe, backlog priorisé) et flexibilité (révision des priorités à chaque sprint, livraisons incrémentales validables par le service RI).

Chaque sprint suit un cycle en quatre étapes : un *Sprint Planning* pour sélectionner et décomposer les user stories, un *Daily Standup* adapté au contexte d'un développeur unique sous forme d'un point quotidien d'avancement et de blocages, une *Sprint Review* pour démontrer les fonctionnalités livrées au service RI, et une *Sprint Retrospective* pour ajuster le processus.

Les rôles sont adaptés au contexte académique : le service RI joue le rôle de *Product Owner* (priorisation du backlog, validation des livraisons), l'encadrante académique assure le rôle de *Scrum Master* (suivi de l'avancement, levée des blocages, orientation technique), et l'étudiante prend en charge l'intégralité du développement. Le suivi du backlog est géré avec GitHub Projects, les user stories correspondant à des issues liées aux pull requests.

3.4.2 Organisation du code : Domain-Driven Design allégé

Le système est structuré selon les principes du *Domain-Driven Design* (DDD), une approche qui consiste à organiser le code autour des domaines métier plutôt qu'autour de considérations techniques.

3.4.2.1 Découpage en domaines métier

Le système est décomposé en domaines indépendants, chacun correspondant à une fonction distincte de la gestion de la mobilité. Ce découpage garantit qu'une modification dans un domaine n'affecte pas les autres.

Domaine	Périmètre
mobility	Conventions de partenariat et quotas
students	Données étudiants et inscriptions
institutions	Universités partenaires
academic	Années académiques et cycle de campagne
reference	Données de référence (pays, départements, niveaux)
incoming	Mobilités entrantes
outgoing	Mobilités sortantes
internships	Stages à l'international
complementary	Formations complémentaires
cti	Données liées aux critères CTI
audit	Traçabilité des actions

TABLE 3.2 – Découpage en domaines métier du système

3.4.2.2 Langage ubiquitaire

Le DDD préconise l'adoption d'un vocabulaire métier commun, partagé entre la développeuse et l'experte du domaine. Les noms des classes et des méthodes reprennent directement le vocabulaire du service RI ce qui facilite les échanges et réduit les risques d'ambiguïté lors de la validation des fonctionnalités.

3.4.2.3 Encapsulation des règles métier

Chaque domaine encapsule ses propres règles métier, ce qui permet de les faire évoluer indépendamment du reste du système. Cette séparation rend le système plus robuste face aux évolutions de besoins, inévitables dans un contexte de développement itératif où certaines règles — comme les critères de priorité dans l'affectation — se précisent au fil des sprints.

3.4.3 Style architectural : architecture en couches

Le système est structuré selon une architecture en couches, où chaque niveau a une responsabilité clairement délimitée. Ce style architectural a été préféré à une architecture microservices, qui aurait été surdimensionnée pour l'échelle du projet et les contraintes d'hébergement de Toulouse INP.

Le système s'articule autour de cinq couches distinctes :

- **La couche présentation** : responsable du rendu des interfaces et de la gestion des interactions utilisateur.
- **La couche applicative** : porte la logique métier, la validation et l'orchestration des domaines.
- **La couche asynchrone** : prend en charge les traitements longs découplés des requêtes, tels que l'algorithme d'affectation ou les synchronisations nocturnes.

- **La couche données** : assure la persistance relationnelle et le stockage des fichiers justificatifs.
- **La couche intégration** : gère la connexion avec les systèmes externes (MoveON, Pégase, Eudonet) via des adaptateurs dédiés.

3.5 Choix du frontend

Cette section présente les besoins spécifiques du frontend, évalue les alternatives disponibles, puis justifie le choix retenu.

3.5.1 Besoins spécifiques

L'interface du projet doit couvrir deux espaces bien distincts : d'un côté, un espace dédié au service RI pour piloter les campagnes, de l'autre, un espace étudiant pour consulter les destinations, suivre sa candidature et interagir avec le chatbot. Certaines pages affichent des données sensibles (résultats d'affectation, GPA) et doivent être construites côté serveur pour éviter qu'elles ne soient accessibles avant que le rôle de l'utilisateur soit vérifié. D'autres, comme le suivi de candidature ou le chatbot, nécessitent une navigation fluide et réactive. Gérer ces deux types de pages dans le même projet, avec une séparation fiable des accès selon le rôle, est le principal critère de choix.

3.5.2 Alternatives évaluées

Trois frameworks ont été évalués au regard des besoins identifiés.

3.5.2.1 Applications monopage — React + Vite

React permet de construire des interfaces entièrement rendues dans le navigateur [9].

Avantages :

- Simple à configurer et à démarrer
- Très bien documenté, avec de nombreuses ressources disponibles en ligne

Limites :

- Tout est chargé et affiché dans le navigateur : les données sensibles comme les affectations ou les GPA y arrivent avant même que le rôle de l'utilisateur soit contrôlé, ce qui fragilise la séparation entre l'espace étudiant et l'espace RI
- La protection des pages selon le rôle doit être codée manuellement, sans mécanisme intégré pour l'imposer au niveau du routing
- Les appels à l'API Django passent tous par le navigateur, ce qui expose les tokens d'authentification au client

3.5.2.2 Angular

Angular est un framework TypeScript développé par Google, utilisé principalement pour des applications d'entreprise avec beaucoup de logique interne [10].

Avantages :

- Architecture très structurée, adaptée aux applications avec de nombreux écrans et règles métier
- Typage TypeScript natif, ce qui réduit les erreurs au moment du développement
- Mécanisme de protection des routes intégré, utile pour séparer les accès selon le rôle utilisateur
- Bien adapté à des interfaces d'administration complexes comme le tableau de bord RI

Limites :

- Pas de rendu côté serveur sans Angular Universal, qui nécessite une configuration supplémentaire non négligeable
- Angular demande un temps de prise en main important : son architecture impose de nombreux concepts spécifiques (modules, décorateurs, injection de dépendances) qui ralentissent le démarrage
- Moins d'exemples disponibles que React/Next.js pour l'intégration avec le SSO CAS utilisé par Toulouse INP

3.5.2.3 Next.js 14

Next.js est un framework basé sur React qui permet de combiner rendu côté serveur et rendu côté client selon les besoins de chaque page [11].

Avantages :

- Rendu hybride : pages sensibles rendues côté serveur, composants interactifs côté client
 - les deux coexistent dans le même projet
- Middleware intégré pour bloquer l'accès aux pages selon le rôle utilisateur avant même leur chargement
- Appels à l'API Django réalisables côté serveur, sans exposer les tokens dans le navigateur
- App Router et React Server Components : chaque composant est rendu côté serveur par défaut, avec des layouts partagés par espace (RI, étudiant) qui structurent naturellement la séparation des interfaces
- TypeScript, Tailwind CSS et intégration SSO CAS/OIDC bien documentés

Limites :

- Configuration initiale un peu plus complexe qu'une application monopage classique

3.5.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des trois solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	React + Vite	Angular	Next.js 14
Rendu côté serveur (SSR)	✗	Partiel	✓
Protection des routes par rôle	Manuelle	Mécanisme intégré	Middleware intégré
Données sensibles côté serveur	✗	✗	✓
TypeScript natif	Optionnel	✓	✓
Intégration SSO CAS/OIDC	✓	Limitée	✓
Rendu hybride SSR + CSR	✗	✗	✓

TABLE 3.3 – Comparaison des frameworks frontend

3.5.4 Décision

Next.js 14 est retenu. Son modèle de React Server Components permet de gérer naturellement les deux espaces du projet : les pages sensibles du tableau de bord RI sont rendues côté serveur avant toute vérification de rôle, tandis que les composants interactifs de l'espace étudiant s'exécutent côté client. C'est le seul framework parmi ceux évalués qui couvre ces deux cas dans le même projet, sans configuration additionnelle complexe.

3.6 Choix du backend

Cette section présente les besoins spécifiques du backend, évalue les alternatives disponibles, puis justifie le choix retenu.

3.6.1 Besoins spécifiques

Le backend de ce projet ne se limite pas à une simple gestion de données. Il doit orchestrer de nombreuses entités métier liées entre elles et gérer des processus parfois complexes, comme le suivi des candidatures, des affectations ou des mobilités étudiantes.

L'application doit également assurer plusieurs fonctions essentielles : gérer les droits d'accès selon les profils (étudiants et administrateurs RI), conserver une trace des actions importantes via un journal d'audit, accompagner l'évolution de la base de données pendant le développement, et s'intégrer au système d'authentification institutionnel CAS/OIDC de Toulouse INP.

Dans ce contexte, le choix du framework ne repose pas uniquement sur les performances, mais aussi sur la qualité des outils qu'il apporte pour structurer, sécuriser et faire évoluer l'application.

3.6.2 Choix du framework backend

3.6.2.1 Alternatives évaluées

Trois frameworks ont été évalués en regard des besoins identifiés précédemment.

3.6.2.1.1 Node.js

Environnement d'exécution JavaScript côté serveur, reconnu pour ses performances sur les applications qui traitent beaucoup de requêtes simultanées [12].

Avantages :

- Bonnes performances sur les traitements non bloquants
- Grand nombre de bibliothèques disponibles
- Gestion native des opérations asynchrones

Limites :

- Aucun outil natif pour la base de données, l'administration ou la gestion des droits : il faut tout assembler manuellement à partir de bibliothèques séparées, ce qui alourdit la configuration et la maintenance.
- Il n'existe pas d'outils suffisamment aboutis pour gérer des mécanismes avancés comme les machines à états ou la traçabilité fine des modifications, pourtant indispensables dans ce projet.
- L'utilisation de Node.js implique de travailler en JavaScript, alors que le reste du système est conçu en Python, ce qui introduit une complexité supplémentaire en termes de cohérence et de maintenance.
- La gestion des évolutions du schéma de base de données au fil des sprints demande davantage d'efforts d'organisation et de suivi, car elle n'est pas aussi intégrée nativement dans l'écosystème.

3.6.2.1.2 FastAPI

Framework Python récent, conçu pour construire des APIs performantes avec une validation automatique des données [13].

Avantages :

- Très bonnes performances grâce au support natif des requêtes asynchrones
- Validation automatique des données entrantes
- Documentation de l'API générée automatiquement

Limites :

- Pas d’outil intégré pour la base de données, l’administration ou la gestion des droits
- Les cycles de vie des entités, le journal d’audit et les permissions seraient à construire de zéro
- L’outil de migration de schéma demande davantage de configuration et d’intervention manuelle, ce qui peut ralentir les cycles d’itération en développement.
- FastAPI est surtout adapté à des APIs légères et rapides, et moins à des applications contenant beaucoup de logique métier.

3.6.2.1.3 Django

Framework Python complet, pensé pour développer rapidement des applications web avec beaucoup de logique interne [14].

Avantages :

- Gestion de la base de données, authentification, permissions et interface d’administration fournis nativement, sans aucune configuration supplémentaire
- `django-fsm` permet de définir des cycles de vie sécurisés sur chaque entité, avec une option qui empêche toute modification directe du statut en dehors des transitions autorisées
- `django-auditlog` enregistre automatiquement toutes les modifications avec l’auteur et la date — indispensable pour les décisions réglementaires
- La gestion des évolutions de schéma est intégrée nativement, ce qui facilite les changements à chaque sprint
- S’intègre facilement avec le SSO CAS/OIDC via des bibliothèques existantes et maintenues

Limites :

- Django était historiquement limité sur les traitements asynchrones, mais cette lacune a été corrigée depuis la version 5
- Légèrement moins performant que FastAPI sur des calculs très complexes, ce qui peut être contourné en passant par des requêtes SQL directes pour les agrégations de durées CTI

3.6.2.2 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des trois solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Node.js	FastAPI	Django 5
ORM et gestion BDD intégrés	✗	✗	✓
Interface d'administration	✗	✗	✓
Machines à états (FSM)	✗	✗	✓
Migrations de schéma	Manuel	nécessite config	✓
Authentification SSO	Manuel	Manuel	✓
Journal d'audit automatique	Manuel	Manuel	✓
Support ASGI / async	✓	✓	✓

TABLE 3.4 – Comparaison des frameworks backend

3.6.2.3 Décision

Django 5 est retenu car le projet repose sur de nombreuses entités liées entre elles, avec des cycles de vie distincts, un journal d'audit obligatoire et une gestion fine des droits entre deux profils utilisateurs.

Utiliser une autre solution aurait impliqué de reconstruire une grande partie de ces mécanismes.

Django permet de disposer de ces fonctionnalités directement, ce qui est un avantage important dans ce projet.

3.6.3 Choix de la couche API

Une fois le framework backend retenu, il reste à choisir la bibliothèque la mieux adaptée pour construire l'API. Cette section évalue les deux solutions compatibles avec Django et justifie le choix retenu.

3.6.3.1 Besoins spécifiques

L'API doit permettre les échanges entre Next.js et Django, valider des structures de données parfois complexes (vœux ordonnés, quotas par département, rapports d'import), générer automatiquement une documentation utilisable par le développeur frontend, et répondre de façon non bloquante aux requêtes qui déclenchent des traitements longs (lancement de l'algorithme, recalcul).

3.6.3.2 Alternatives évaluées

Deux solutions ont été évaluées pour exposer l'API entre le frontend Next.js et le backend Django.

3.6.3.2.1 Django REST Framework (DRF)

Django REST Framework est la bibliothèque la plus utilisée pour construire des APIs avec

Django. Elle fournit un ensemble complet d'outils pour la sérialisation, la validation et la gestion des permissions [15].

Avantages :

- Très mature et bien documenté, avec une grande communauté
- Outillage complet : gestion des tokens JWT, filtres, pagination, génération de documentation via `drf-spectacular`
- Fonctionne parfaitement avec l'ORM et les permissions Django

Limites :

- Le support des requêtes asynchrones est partiel, ce qui pose problème pour les endpoints qui déclenchent le lancement de l'algorithme ou les synchronisations avec MoveON, Pégase et Eudonet
- Les sérialiseurs DRF demandent beaucoup de code répétitif, surtout quand on travaille avec plusieurs entités
- La documentation de l'API nécessite une configuration externe séparée, à maintenir en parallèle

3.6.3.2.2 Django Ninja

Framework API pour Django, basé sur une validation des données structurée et typée via Pydantic [16].

Avantages :

- Toutes les routes supportent nativement les traitements asynchrones : un endpoint peut lancer un calcul ou déclencher une synchronisation sans bloquer le serveur, en déléguant l'exécution à un système de traitement en arrière-plan.
- La validation des données via Pydantic est plus concise que DRF
- La documentation de l'API est générée automatiquement, sans aucune configuration supplémentaire
- Conserve entièrement l'écosystème Django : ORM, administration, cycles de vie, permissions

Limites :

- Plus récent que DRF, avec moins d'extensions disponibles
- Moins de ressources en ligne pour des cas très spécifiques

3.6.3.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des deux solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	DRF	Django Ninja
Support async natif	Partiel	✓
Validation Pydantic (typage strict)	✗	✓
Documentation auto-générée	✗	✓
Intégration ORM Django	✓	✓
Maturité et communauté	✓	Partiel
Réduction du code répétitif	✗	✓

TABLE 3.5 – Comparaison DRF et Django Ninja

3.6.3.4 Décision

Django Ninja est retenu. Les endpoints les plus critiques du projet — déclenchement de l’algorithme, recalcul, synchronisations ETL — doivent répondre immédiatement sans bloquer l’interface pendant le traitement. Django Ninja gère cela nativement, là où DRF aurait demandé des contournements. La validation Pydantic réduit aussi significativement le code répétitif sur plusieurs entités.

La stack backend retenue est donc : **Django 5 + Django Ninja**.

3.7 Choix de la base de données

Cette section présente les besoins spécifiques liés au stockage des données, évalue les alternatives disponibles, puis justifie le choix retenu.

3.7.1 Besoins spécifiques

La base de données doit supporter un modèle avec plusieurs entités reliées entre elles par de nombreuses clés étrangères, stocker les réponses brutes des APIs externes avant transformation (`RawImport`), et garantir la cohérence des données lors des opérations qui touchent plusieurs entités à la fois (transitions de cycle de vie, affectations, journal d’audit).

3.7.2 Alternatives évaluées

Trois systèmes de gestion de base de données ont été évalués en regard des besoins identifiés précédemment.

3.7.2.1 MySQL

MySQL est un système de gestion de base de données relationnelle très répandu, bien supporté par l’écosystème Django et largement utilisé en production [17].

Avantages :

- Système très répandu, avec une grande communauté et une documentation abondante
- Bonne intégration avec l'écosystème Django
- Largement utilisé en production, avec des outils de sauvegarde matures

Limites :

- Support moins adapté pour le stockage et l'exploitation de données au format JSON, notamment pour les imports bruts (`RawImport`)
- Fonctionnalités limitées pour gérer des mécanismes de file d'attente de tâches
- Moins adapté aux opérations transactionnelles complexes impliquant plusieurs entités liées

3.7.2.2 MongoDB

MongoDB est une base de données orientée documents, conçue pour offrir une grande flexibilité dans la structure des données et particulièrement adaptée au stockage JSON [18].

Avantages :

- Grande flexibilité dans la structure des données
- Adapté au stockage de documents JSON
- Bonnes performances sur les lectures de documents non structurés

Limites :

- Le modèle de données du projet repose fortement sur des relations entre entités (quotas, affectations, mobilités), ce qui rend l'absence de relations structurées moins adaptée
- Intégration limitée avec les outils ORM utilisés dans le projet
- Absence de support direct pour certains mécanismes de traitement asynchrone utilisés dans l'architecture

3.7.2.3 PostgreSQL 16

PostgreSQL est un système de gestion de base de données relationnelle open source, reconnu pour sa robustesse, la richesse de ses fonctionnalités et son respect strict des standards SQL [19].

Avantages :

- Gestion robuste des transactions, adaptée aux opérations impliquant plusieurs entités liées
- Support natif du format JSON avec indexation, utile pour le stockage des données brutes (`RawImport`)
- Possibilité d'utiliser la base comme support de file d'attente pour les traitements asynchrones

- Bon niveau d'intégration avec les outils de développement du projet
- Outils de sauvegarde et de restauration fiables

Limites :

- Mise en place initiale plus complète, nécessitant une configuration structurée dès le début du projet

3.7.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des trois solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	MySQL / MariaDB	MongoDB	PostgreSQL 16
Transactions ACID multi-tables	✓	Partiel	✓
Support JSONB natif + indexation	Partiel	✓	✓
File d'attente de tâches	✗	✗	✓
Intégration ORM Django	✓	Partiel	✓
Extension pgcrypto (chiffrement)	✗	✗	✓
Outils de sauvegarde fiables	✓	✓	✓

TABLE 3.6 – Comparaison des systèmes de gestion de base de données

3.7.4 Décision

PostgreSQL 16 est retenu. Il répond de manière cohérente à l'ensemble des besoins identifiés — gestion des relations, support du JSON natif, traitements asynchrones et chiffrement — dans un seul service, sans introduire de composant supplémentaire dans l'infrastructure.

3.8 Choix du format de stockage des imports bruts

Cette section précise comment sont stockées les réponses brutes des APIs externes avant leur traitement, en évaluant les alternatives disponibles et en justifiant le format retenu.

3.8.1 Besoins spécifiques

L'entité `RawImport` doit conserver la réponse complète de chaque appel d'API externe (MoveON, Pégase, Eudonet) avant toute transformation. Ces données sont semi-structurées

et variables d’une source à l’autre. Le format retenu doit permettre de relire, d’interroger et de tracer ces imports tout en restant cohérent avec le reste du modèle de données.

3.8.2 Alternatives évaluées

3.8.2.1 Fichiers plats (JSON sur le système de fichiers)

Un fichier plat est un fichier texte brut qui stocke des données sans structure hiérarchique ni mécanisme d’indexation. Dans ce contexte, chaque réponse d’API serait sérialisée en JSON et enregistrée dans un répertoire dédié sur le serveur, un fichier par import.

Avantages :

- Mise en place immédiate, sans configuration particulière
- Lecture directe par n’importe quel éditeur de texte

Limites :

- Aucune possibilité d’interroger le contenu avec des requêtes SQL
- Intégrité référentielle avec les autres entités non garantie
- Gestion des accès concurrents et des transactions non assurée
- Sauvegarde dissociée du reste de la base de données

3.8.2.2 Base NoSQL dédiée (MongoDB)

Les imports bruts sont stockés dans une base documentaire séparée, spécialisée dans le stockage de documents JSON.

Avantages :

- Très bien adapté au stockage de documents JSON hétérogènes
- Bonne performance sur les lectures de documents non structurés

Limites :

- Service supplémentaire à déployer, superviser et sauvegarder
- Aucune relation possible avec les entités de la base relationnelle
- Surcoût d’infrastructure injustifié pour ce seul cas d’usage

3.8.2.3 Type JSONB dans PostgreSQL

Les réponses brutes sont stockées dans une colonne de type **JSONB** au sein de la table **RawImport**, dans la même base PostgreSQL.

Avantages :

- Stockage dans la base existante, sans service supplémentaire
- Indexation du contenu JSON pour des requêtes efficaces

- Jointures possibles avec les autres tables du modèle
- Mêmes garanties ACID et même périmètre de sauvegarde que le reste

Limites :

- Moins flexible que MongoDB pour des structures JSON très hétérogènes, ce qui reste sans impact ici compte tenu du faible nombre de sources

3.8.3 Synthèse comparative

Critère	Fichiers plats	MongoDB	JSONB (PostgreSQL)
Requêtes SQL sur le contenu	✗	✗	✓
Intégrité référentielle	✗	✗	✓
Garanties ACID	✗	Partiel	✓
Aucun service supplémentaire	✓	✗	✓
Indexation du contenu JSON	✗	✓	✓
Sauvegarde unifiée	✗	✗	✓

TABLE 3.7 – Comparaison des formats de stockage pour `RawImport`

3.8.4 Décision

Le type **JSONB** de PostgreSQL est retenu pour le stockage des imports bruts. Il évite toute fragmentation de l'infrastructure en réutilisant la base déjà en place, tout en offrant les capacités d'interrogation nécessaires sur les données semi-structurées. Les fichiers plats ne permettent pas de garantir la cohérence avec le reste du modèle, et l'ajout de MongoDB n'apporterait aucun avantage fonctionnel pour ce seul cas d'usage.

3.9 Choix du gestionnaire de tâches asynchrones

Cette section présente les besoins en traitement asynchrone du projet, évalue les solutions disponibles, puis justifie le choix retenu.

3.9.1 Besoins spécifiques

Certaines opérations du système ne peuvent pas s'exécuter de façon synchrone sans bloquer l'interface utilisateur. Deux catégories de tâches ont été identifiées.

La première catégorie regroupe les tâches déclenchées par des événements : le lancement de l'algorithme d'affectation, le recalcul des durées après modification d'un état, et le traitement des imports qui peuvent durer plusieurs secondes.

La seconde catégorie regroupe les tâches planifiées : les synchronisations nocturnes avec MoveON, Pégase et Eudonet, ainsi que la suppression automatique des imports bruts anciens.

Le volume reste modéré — quelques dizaines de tâches par jour en fonctionnement normal, quelques centaines lors des périodes de traitement — sans besoin de traitement intensif en continu.

3.9.2 Alternatives évaluées

Deux solutions ont été évaluées pour la gestion des tâches asynchrones dans l'écosystème Django.

3.9.2.1 Celery + Redis

Celery est la bibliothèque Python de référence pour l'exécution de tâches en arrière-plan. Elle s'appuie sur un broker de messages externe — généralement Redis ou RabbitMQ — pour acheminer les tâches vers les workers [20] [21].

Avantages :

- Très mature, bien documenté, avec une grande communauté
- Capable de traiter un très grand volume de tâches en parallèle
- Compatible avec plusieurs brokers (Redis, RabbitMQ)
- Outil de supervision dédié (Flower)

Limites :

- Trois services supplémentaires à déployer et maintenir : le worker Celery, le planificateur Celery Beat et Redis
- Redis consomme entre 50 et 200 Mo de mémoire pour un rôle que PostgreSQL, déjà présent, peut assurer seul
- Complexité disproportionnée par rapport au volume réel du projet

3.9.2.2 Django-Q2 + PostgreSQL

Django-Q2 est une bibliothèque de gestion de tâches asynchrones conçue spécifiquement pour Django. Elle utilise directement PostgreSQL comme file d'attente, sans nécessiter de broker externe [22].

Avantages :

- Aucun service supplémentaire : PostgreSQL joue le rôle de file d'attente sans configuration additionnelle
- Planificateur de tâches intégré, adapté aux synchronisations nocturnes et aux suppressions programmées

- Supervision accessible depuis l’interface d’administration Django existante, sans outils tiers
- Seulement deux services à opérer au lieu de cinq avec Celery

Limites :

- Moins performant que Celery pour des volumes très élevés, sans incidence pour ce projet
- Communauté plus restreinte que Celery

3.9.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des deux solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Celery + Redis	Django-Q2 + PostgreSQL
Aucun service supplémentaire	✗	✓
Planificateur intégré	Partiel	✓
Supervision via admin Django	✗	✓
Adapté au volume du projet	Partiel	✓
Maturité et communauté	✓	Partiel
Intégration native avec Django	Partiel	✓

TABLE 3.8 – Comparaison des gestionnaires de tâches asynchrones

3.9.4 Décision

Django-Q2 avec PostgreSQL est retenu. Il réutilise la base de données déjà en place comme file d’attente, ce qui évite d’introduire un service supplémentaire dans l’infrastructure. Le volume de tâches du projet ne justifie pas la complexité opérationnelle de Celery. La supervision des traitements depuis l’interface d’administration Django existante simplifie également le suivi quotidien pour le service RI.

3.10 Choix du stockage de fichiers

Cette section présente les besoins en stockage de fichiers du projet, évalue les solutions disponibles, puis justifie le choix retenu.

3.10.1 Besoins spécifiques

Le projet requiert un mécanisme de dépôt et de stockage de fichiers pour les justificatifs soumis par les étudiants lors de la déclaration de mobilités complémentaires. Ces fichiers

doivent être stockés de manière sécurisée et chiffrée, accessibles uniquement par le service RI et l'étudiant concerné, avec une suppression automatique conforme au RGPD.

3.10.2 Alternatives évaluées

Deux approches ont été évaluées pour répondre à ces besoins.

3.10.2.1 Système de fichiers local

Le stockage local consiste à enregistrer les fichiers directement sur le système de fichiers du serveur hébergeant l'application Django, dans un répertoire dédié.

Avantages :

- Simple à configurer, sans dépendance supplémentaire

Limites :

- Aucun chiffrement natif des fichiers stockés
- Non portable entre environnements (développement, staging, production)
- Difficile à sauvegarder et à migrer indépendamment de la base de données

3.10.2.2 MinIO (stockage objet S3-compatible)

MinIO est une solution de stockage objet open source, auto-hébergeable et compatible avec l'API S3 d'Amazon [23]. Elle s'intègre nativement dans une infrastructure Docker et ne nécessite aucun service cloud externe.

Avantages :

- Stockage objet auto-hébergé, compatible avec l'API S3 standard
- Chiffrement côté serveur (SSE-S3) des fichiers stockés
- Génération d'URLs pré-signées pour un accès temporaire et sécurisé, sans exposer les fichiers directement
- Déployable comme container Docker interne, sans aucune dépendance externe ni service cloud
- Politique de rétention et suppression automatique configurables, conformes aux exigences RGPD

Limites :

- Configuration initiale plus complexe qu'un stockage local

3.10.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des deux solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Fichiers local	MinIO
Chiffrement natif (SSE-S3)	✗	✓
URLs pré-signées (accès sécurisé)	✗	✓
Portabilité multi-environnements	✗	✓
Suppression automatique (RGPD)	✗	✓
Déploiement Docker intégré	✗	✓
Simplicité de configuration	✓	Partiel

TABLE 3.9 – Comparaison des solutions de stockage de fichiers

3.10.4 Décision

MinIO est retenu. MinIO peut être déployé selon deux modes : **standalone** (instance unique sur un seul serveur) ou **distributed** (plusieurs nœuds pour la haute disponibilité et la tolérance aux pannes). Le mode distribué est destiné aux infrastructures multi-serveurs nécessitant une réplication des données.

Dans le cadre de ce projet, le déploiement cible un **serveur unique on-premise** au sein de l'établissement. Le mode standalone est donc retenu : il correspond exactement à cette topologie, sans surcharger l'infrastructure avec des mécanismes de distribution inutiles. MinIO répond ainsi directement aux exigences de sécurité (chiffrement SSE-S3) et de conformité RGPD (rétention et suppression automatiques), tout en s'intégrant de façon transparente dans Docker Compose sans aucun service externe.

3.11 Couche de routage des requêtes

Cette section présente les besoins en matière de routage réseau, évalue les deux approches disponibles, puis justifie le choix retenu.

3.11.1 Besoins spécifiques

Le système doit disposer d'un composant chargé de recevoir les requêtes HTTP entrantes, d'assurer la terminaison SSL, de router le trafic vers le frontend Next.js et l'API Django Ninja, et de servir directement les fichiers statiques sans les faire transiter par l'application. Ce composant sera administré par les équipes IT de Toulouse INP ; sa configuration doit donc être lisible et maintenable sans expertise Docker particulière.

3.11.2 Alternatives évaluées

Deux approches ont été évaluées pour assurer ce rôle.

3.11.2.1 Reverse proxy

Un reverse proxy est un composant qui se place entre les clients et les services internes. Il reçoit les requêtes HTTP entrantes, assure la terminaison SSL, les achemine vers le service approprié selon des règles de routage, et sert les fichiers statiques directement sans solliciter l'application.

Avantages :

- Composant léger, dédié au routage et à la terminaison SSL
- Configuration statique (Nginx) ou dynamique (Traefik) selon le besoin
- Bien adapté à une architecture monolithique sur un seul serveur
- Maturité et documentation abondantes pour les deux outils

Limites :

- Ne centralise pas l'authentification ni le contrôle de débit (ce qui n'est pas requis ici)

3.11.2.2 API Gateway

Une API Gateway est un composant qui, en plus du routage, centralise la gestion de l'authentification, du contrôle de débit et de la transformation des requêtes. Elle est conçue pour exposer plusieurs microservices à des clients externes variés.

Avantages :

- Gestion centralisée de l'authentification et du contrôle de débit
- Adapté aux architectures microservices exposées à l'externe

Limites :

- Complexité et surcoût injustifiés pour une architecture déployée sur un seul serveur
- Le projet consomme des APIs externes, il n'en expose pas à des clients variés — ce cas d'usage ne s'applique pas

3.11.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des deux approches, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Reverse proxy	API Gateway
Adapté à un seul serveur	✓	✗
Terminaison SSL	✓	✓
Routage vers services internes	✓	✓
Gestion auth. centralisée	✗	✓
Complexité d'exploitation	Faible	Élevée
Pertinent pour ce projet	✓	✗

TABLE 3.10 – Comparaison des approches de routage des requêtes

3.11.4 Décision

Le **reverse proxy** est retenu. L'architecture est déployée sur un seul serveur et le système consomme des APIs externes sans en exposer à des clients variés. Une API Gateway introduirait une complexité sans aucun bénéfice fonctionnel dans ce contexte.

3.11.5 Choix de l'implémentation : Nginx vs Traefik

Une fois le reverse proxy retenu, il reste à choisir l'outil d'implémentation. Cette partie évalue les deux solutions les plus utilisées dans l'écosystème Docker et justifie le choix retenu.

3.11.5.1 Besoins spécifiques

L'outil retenu doit assurer la terminaison SSL, le routage vers le frontend Next.js et l'API Django Ninja, et la distribution des fichiers statiques. Sa configuration doit être lisible et auditable par les équipes IT de Toulouse INP qui administreront le serveur, sans expertise Docker avancée.

3.11.5.2 Alternatives évaluées

Deux outils open source ont été comparés pour implémenter le reverse proxy.

3.11.5.2.1 Nginx

Nginx est un serveur web et reverse proxy très répandu, reconnu pour sa stabilité et la lisibilité de ses fichiers de configuration [24].

Avantages :

- Très stable et bien documenté, avec un comportement prévisible
- Configuration dans des fichiers texte simples, lisibles et faciles à auditer par n'importe quel administrateur système
- Gestion SSL/TLS fiable, excellentes performances sur le routage des requêtes

Limites :

- La gestion des certificats SSL nécessite un outil externe (Certbot)
- Configuration manuelle à mettre à jour lors de changements de services

3.11.5.2.2 Traefik

Traefik est un reverse proxy conçu pour les environnements conteneurisés. Il se configure dynamiquement à partir des labels des conteneurs Docker actifs [25].

Avantages :

- Configuration automatique à partir des conteneurs Docker actifs
- Gestion native des certificats SSL via Let's Encrypt
- Interface de supervision intégrée

Limites :

- Configuration déclarative moins lisible pour des équipes IT non familières avec Docker
- Comportement moins prévisible dans un environnement institutionnel avec des contraintes réseau spécifiques

3.11.5.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacun des deux outils, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Nginx	Traefik
Configuration lisible (fichiers texte)	✓	Partiel
Gestion SSL native	Partiel	✓
Automatisation Docker	Partiel	✓
Prévisibilité en environnement institutionnel	✓	Partiel
Maintenabilité par équipes IT générales	✓	✗
Maturité et documentation	✓	✓

TABLE 3.11 – Comparaison des outils de reverse proxy

3.11.5.4 Décision

Nginx est retenu. Le serveur sera administré par les équipes IT de Toulouse INP. Disposer d'une configuration statique dans des fichiers texte qu'un administrateur système peut lire et modifier sans expertise Docker est, dans ce contexte institutionnel, plus important que l'automatisation dynamique proposée par Traefik.

3.12 Stratégie de performance base de données

Cette section présente les enjeux de performance liés à la croissance des données, évalue les deux approches disponibles, puis justifie la stratégie retenue.

3.12.1 Contexte et enjeux

Plusieurs tables du projet — mobilités, affectations, imports — grossissent d'une campagne à l'autre. Le système doit pouvoir les filtrer rapidement par année universitaire, que ce soit pour afficher le tableau de bord RI, calculer les durées CTI ou lancer l'algorithme d'affectation. Deux approches ont été évaluées pour répondre à cet enjeu.

3.12.2 Alternatives évaluées

3.12.2.1 Partitionnement par année

Le partitionnement consiste à découper physiquement une table en sous-tables, une par valeur de la colonne de partition — ici, par année universitaire. PostgreSQL ne lit alors que la partition concernée lors d’une requête filtrée.

Avantages :

- Réduction significative du volume scanné pour les requêtes filtrées par année
- Efficace lorsque les tables atteignent plusieurs millions de lignes

Limites :

- La colonne de partitionnement doit être incluse dans toutes les contraintes d’unicité, ce qui peut créer des conflits avec les migrations Django au fil des sprints
- Complexité supplémentaire du schéma pour un volume qui ne le justifie pas

3.12.2.2 Indexation ciblée

L’indexation ciblée consiste à créer des index sur les colonnes les plus utilisées dans les filtres, sans modifier la structure physique des tables.

Avantages :

- Aucune modification du schéma ni des migrations Django
- Index déclarés directement dans les modèles, visibles dans le code et versionnés avec lui
- Transparent pour l’ORM, sans configuration PostgreSQL séparée

Limites :

- Moins efficace que le partitionnement pour des volumes de plusieurs millions de lignes
 - sans incidence ici

3.12.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des deux approches, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Partitionnement	Indexation ciblée
Adapté au volume du projet	✗	✓
Compatible avec les migrations Django	Partiel	✓
Déclaration dans les modèles	✗	✓
Aucune configuration PostgreSQL séparée	✗	✓
Efficacité à très grande échelle	✓	Partiel

TABLE 3.12 – Comparaison des stratégies de performance base de données

3.12.4 Décision

L'**indexation ciblée** est retenue. L'ENSEEIHТ traite quelques centaines d'étudiants en mobilité par an. Même en projetant ce volume sur dix ans, les tables restent très loin du seuil où le partitionnement apporterait un gain mesurable. Des index sont créés sur `academic_year_id` et sur la combinaison (`academic_year_id`, `student_id`) pour les tables les plus sollicitées, directement dans les modèles Django concernés.

3.13 Choix de l'algorithme d'affectation

Cette section présente les contraintes auxquelles l'algorithme doit répondre, évalue les approches disponibles, puis justifie le choix retenu.

3.13.1 Besoins spécifiques

L'algorithme doit attribuer à chaque étudiant une destination parmi ses vœux, en respectant un ensemble de contraintes institutionnelles. Les résultats ont un impact réel sur les parcours : le service RI doit pouvoir justifier chaque affectation. L'équité entre candidats et la reproductibilité des résultats sont des exigences essentielles.

Le classement des étudiants repose principalement sur le GPA, complété par des règles institutionnelles. Les affectations doivent respecter les quotas par accord de mobilité et par département, les contraintes de financement, ainsi que la situation des étudiants étrangers dans le classement global. Chaque étudiant exprime jusqu'à trois vœux ordonnés ; l'algorithme traite les vœux dans l'ordre et recherche une affectation alternative en cas d'indisponibilité.

3.13.2 Alternatives évaluées

Deux approches algorithmiques ont été évaluées pour répondre à ces besoins.

3.13.2.1 Algorithme glouton

L'algorithme glouton parcourt les candidats dans l'ordre de leur GPA et affecte chacun à son meilleur vœu encore disponible, sans jamais revenir sur une décision déjà prise.

Avantages :

- Simple à implémenter et à expliquer
- Résultat prévisible, complexité $O(n^2)$

Limites :

- Le résultat dépend de l'ordre de traitement : deux étudiants avec des GPA très proches peuvent se retrouver dans des situations très différentes selon leur position dans la liste
- Une légère variation de l'ordre d'entrée peut produire des résultats différents, ce qui est incompatible avec l'exigence de reproductibilité

3.13.2.2 Gale-Shapley

L'algorithme de Gale-Shapley est un algorithme d'affectation stable qui procède par rondes successives de propositions et de rejets différés jusqu'à convergence. Il garantit qu'aucune paire étudiant-destination ne peut former une meilleure association que celle retenue à l'issue du processus [GaleShapley1962].

Avantages :

- Résultat stable : il est mathématiquement impossible qu'un étudiant et une destination forment une meilleure paire que celle retenue, quelles que soient les données
- Équitable : le résultat ne dépend pas de l'ordre de traitement, deux exécutions avec les mêmes données donnent toujours le même résultat
- Optimal pour les étudiants dans la variante Student-Proposing : chacun obtient la meilleure destination possible compte tenu de son classement et de ses vœux
- Complexité $O(n^2)$, tout à fait adaptée à quelques centaines d'étudiants

Limites :

- Légèrement plus complexe à implémenter que l'algorithme glouton

3.13.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des deux approches, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Algo. glouton	Gale-Shapley
Stabilité des affectations	✗	✓
Reproductibilité	✗	✓
Indépendance à l'ordre de traitement	✗	✓
Optimalité pour les étudiants	Partiel	✓
Explicabilité des résultats	Partiel	✓
Simplicité d'implémentation	✓	Partiel

TABLE 3.13 – Comparaison des algorithmes d'affectation

3.13.4 Décision

Gale-Shapley est retenu, dans sa variante **Student-Proposing** : ce sont les étudiants qui font des propositions aux destinations, et non l'inverse. Cette variante garantit que chaque étudiant obtient la meilleure destination possible compte tenu de son classement et de ses vœux — propriété dite d'*optimalité côté proposant*.

Les contraintes de quota sont vérifiées à deux niveaux simultanément à chaque proposition : un quota global par accord bilatéral (capacité de l'université partenaire) et un quota local par département pédagogique (places allouées au sein de l'ENSEEIH). Une destination est disponible seulement si les deux niveaux sont respectés ; sinon la proposition est rejetée et l'étudiant passe à son vœu suivant.

L'algorithme glouton est écarté car il est sensible à l'ordre de traitement et ne garantit pas la reproductibilité. Gale-Shapley assure un résultat stable, équitable et reproductible dans ce cadre multi-contraint.

3.14 Choix du moteur de recommandation

Cette section présente les besoins du module de recommandation, évalue les approches disponibles, puis justifie la méthode retenue.

3.14.1 Besoins spécifiques

Le moteur de recommandation a pour objectif d'aider les étudiants à choisir leurs destinations avant la saisie des vœux dans MoveON. Il ne prend pas de décision, mais propose un classement des destinations selon leur pertinence pour le profil de l'étudiant. Les recommandations doivent s'appuyer sur le profil académique et sur les données historiques disponibles, tout en restant interprétables pour l'étudiant.

3.14.2 Alternatives évaluées

Trois grandes familles d'approches ont été évaluées pour répondre à ce besoin.

3.14.2.1 Approche basée sur des règles (rule-based)

Cette approche repose sur des règles métier simples définies manuellement, telles que filtrer les destinations selon le département et classer par taux d'acceptation historique.

Avantages :

- Simple à mettre en place, sans données d'apprentissage
- Facile à comprendre et à expliquer

Limites :

- Peu flexible, ne s'adapte pas aux profils individuels
- Résultats souvent trop génériques

3.14.2.2 Approche par similarité (collaborative filtering)

Cette approche propose des destinations en se basant sur les choix d'étudiants ayant des profils similaires dans les promotions précédentes.

Avantages :

- Prend en compte les expériences passées des promotions
- Permet des recommandations plus personnalisées

Limites :

- Nécessite un volume important de données historiques
- Moins efficace pour les nouvelles destinations
- Mise en œuvre plus complexe

3.14.2.3 Approche par apprentissage supervisé

Cette approche consiste à entraîner un modèle pour prédire une probabilité de compatibilité entre un étudiant et une destination. Deux modèles ont été comparés au sein de cette famille.

3.14.2.3.1 Régression logistique

La régression logistique est un modèle statistique qui estime la probabilité d'un résultat à partir de variables d'entrée numériques. Elle est particulièrement adaptée aux volumes de données modérés [26].

Avantages :

- Modèle simple et rapide à entraîner
- Résultats faciles à interpréter et à expliquer
- Adapté à des volumes de données modérés

Limites :

- Capture difficilement des relations complexes entre variables

3.14.2.3.2 XGBoost

XGBoost est un algorithme de gradient boosting qui construit un ensemble d'arbres de décision de façon séquentielle pour améliorer la précision prédictive [26].

Avantages :

- Capable de modéliser des relations complexes entre variables
- Très bonnes performances prédictives

Limites :

- Plus complexe à entraîner et à maintenir
- Résultats moins interprétables
- Risque de surapprentissage avec un volume de données limité

3.14.3 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des approches, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Règles	Collab.	Régression log.	XGBoost
Personnalisation	✗	✓	✓	✓
Volume de données requis	✓	✗	Partiel	Partiel
Interprétabilité	✓	Partiel	✓	✗
Risque de surapprentissage	✓	Partiel	✓	✗
Simplicité de maintenance	✓	✗	✓	✗

TABLE 3.14 – Comparaison des approches de recommandation

3.14.4 Décision

L'apprentissage supervisé par régression logistique est retenu. Le modèle estime, pour chaque couple étudiant–destination, une probabilité de compatibilité à partir des variables suivantes :

Variable	Description
GPA de l'étudiant	Note académique normalisée, principal signal de classement
Département pédagogique	Filtrage sur les accords ouverts au département de l'étudiant
Destination	Identifiant de l'université partenaire
Historique des affectations	Taux de sélection par destination sur les trois dernières années

TABLE 3.15 – Variables d'entrée du moteur de recommandation

Les destinations sont classées par ordre de probabilité décroissante afin de proposer une liste de recommandations personnalisées.

La régression logistique est retenue car elle offre un bon compromis entre simplicité, interprétabilité et facilité de maintenance. Le volume de données — environ 900 observations sur trois ans — ne justifie pas XGBoost, dont les performances supplémentaires s'accompagnent d'un risque accru de surapprentissage.

Risque de démarrage à froid. Certaines destinations rares ou récentes disposent de peu d'observations historiques. Pour ces cas, le modèle se rabat sur le GPA et le département comme seuls signaux, ce qui reste cohérent avec les règles de sélection. Le module sera activé à partir de la deuxième année de fonctionnement, une fois qu'un historique minimal d'une campagne complète aura été constitué.

3.15 Choix de l'orchestration RAG

Cette section présente les besoins du module RAG et le modèle de langage imposé, évalue les frameworks d'orchestration disponibles, puis justifie le choix retenu.

3.15.1 Besoins spécifiques

Le composant RAG du projet est un chatbot destiné aux étudiants, capable de répondre à des questions sur les procédures de mobilité à partir de la documentation institutionnelle de l'ENSEEIH : guides, critères d'éligibilité, délais et démarches administratives. Le cas d'usage est relativement simple : une base documentaire stable, des questions en français ou en anglais, et des réponses ancrées dans les documents officiels.

3.15.2 Modèle de langage

Le modèle de langage retenu est **Mistral** [27] (`mistral-small-latest`), conformément aux exigences de l'ENSEEIH. Il est accessible via l'**API Mistral ILAS**, une infrastructure nationale dédiée aux établissements de l'enseignement supérieur français, hébergée sur des serveurs localisés en France.

Ce mode d'accès institutionnel répond directement aux exigences RGPD : les données traitées restent dans l'infrastructure nationale et ne sont pas transmises à des tiers commerciaux. Il est néanmoins impératif que les données personnelles des étudiants ne soient jamais incluses dans les requêtes envoyées au modèle — les questions au chatbot sont transmises telles que saisies, sans enrichissement avec des données de dossier.

3.15.3 Alternatives évaluées

Quatre approches ont été évaluées pour orchestrer le pipeline RAG autour du modèle Mistral.

3.15.3.1 LangChain

LangChain est un framework d'orchestration IA généraliste, compatible avec un large éventail de modèles et de bases vectorielles [28].

Avantages :

- Grande communauté et nombreuses ressources disponibles
- Intégration simple avec Mistral et ChromaDB
- Adapté aux pipelines IA complexes et multi-étapes

Limites :

- API qui évolue fréquemment, induisant des risques de rupture
- Nombreuses dépendances, surcoût pour un chatbot documentaire simple

3.15.3.2 LlamaIndex

LlamaIndex est un framework spécialisé dans la recherche et la génération à partir de documents structurés [29].

Avantages :

- Conçu spécifiquement pour les applications RAG documentaires
- Bonne intégration avec Mistral et les bases vectorielles
- API plus stable, prise en main plus directe

Limites :

- Moins adapté aux architectures IA multi-agents très complexes

3.15.3.3 Haystack

Haystack est un framework orienté production pour les systèmes de recherche et de génération augmentée [30].

Avantages :

- Architecture modulaire adaptée aux déploiements en production
- Compatible avec Mistral

Limites :

- Documentation moins accessible pour des cas d’usage simples
- Intégration avec Mistral moins directe que LlamaIndex

3.15.3.4 Approche directe (sans framework)

L’approche directe consiste à implémenter manuellement le pipeline RAG via des appels à l’API Mistral, sans couche d’abstraction intermédiaire.

Avantages :

- Contrôle total sur chaque étape du pipeline
- Peu de dépendances externes

Limites :

- Nécessite de développer manuellement toute la logique RAG (chunking, indexation, retrieval, construction du prompt)
- Maintenance plus complexe à long terme

3.15.4 Synthèse comparative

Le tableau suivant récapitule les critères évalués pour chacune des quatre approches, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	LangChain	LlamaIndex	Haystack	Direct
Spécialisé RAG	Partiel	✓	Partiel	✗
Intégration Mistral	✓	✓	Partiel	✓
Stabilité de l’API	✗	✓	✓	✓
Adapté à un cas simple	Partiel	✓	Partiel	Partiel
Maintenance à long terme	Partiel	✓	✓	✗

TABLE 3.16 – Comparaison des frameworks d’orchestration RAG

3.15.5 Décision

LlamaIndex est retenu comme framework d’orchestration RAG, avec **ChromaDB** comme base vectorielle et **Mistral** comme modèle de langage. Cette combinaison correspond directement au besoin du projet : construire un chatbot documentaire simple, basé sur une base de connaissances stable. LlamaIndex fournit les fonctionnalités nécessaires sans surcoût de complexité, avec une API stable et une intégration directe avec Mistral et ChromaDB.

3.16 Synthèse des choix technologiques

TABLE 3.17 – Synthèse des choix technologiques

Composant	Technologie retenue	Justification principale
Méthodologie de développement	Scrum	Développement itératif et adaptation progressive aux besoins du service RI
Frontend	Next.js 14	Rendu hybride SSR/CSR, séparation sécurisée des espaces RI et étudiant
Backend framework	Django 5	ORM, administration, gestion des droits et logique métier intégrés
Couche API	Django Ninja	Support natif de l'asynchrone, validation Pydantic, documentation automatique
Base de données	PostgreSQL 16	Transactions robustes, support JSON et bonne intégration avec Django
Gestionnaire de tâches asynchrones	Django-Q2 + PostgreSQL	Aucun service supplémentaire et supervision intégrée à Django
Reverse proxy	Nginx	Configuration simple, stable et adaptée à un environnement institutionnel
Stratégie de performance BDD	Indexation ciblée	Solution simple et suffisante pour le volume de données du projet
Algorithme d'affectation	Gale-Shapley	Stabilité, équité et reproductibilité des affectations
Moteur de recommandation	Régression logistique	Modèle simple, interprétable et adapté à un volume de données limité
Orchestration RAG	LlamaIndex + ChromaDB	Solution adaptée à un chatbot documentaire simple et stable
Modèle de langage	Mistral	Compatible avec le besoin multilingue du chatbot étudiant
Conteneurisation	Docker + Docker Compose	Déploiement reproductible et simplification de l'environnement

Composant	Technologie retenue	Justification principale
Gestion des états métier	django-fsm	Transitions sécurisées et intégration directe avec Django ORM
Journalisation et audit	django-auditlog	Historique automatique des modifications et traçabilité
Stockage fichiers	MinIO (S3-compatible)	Stockage objet auto-hébergé, chiffrement SSE-S3, conforme RGPD, intégré comme container Docker sans service externe.
Authentification	CAS/OIDC	Exigence institutionnelle — SSO Toulouse INP avec détection automatique du rôle depuis le token.
Monitoring d'erreurs	Sentry	Capture automatique des exceptions backend et frontend avec contexte de la requête
Métriques système	Prometheus + Grafana	Suivi des métriques de performance (CPU, mémoire, temps de réponse) sur l'infrastructure de production
Tests de charge	Locust	Simulation de trafic concurrent pour valider les cibles de performance (200 ms / 50 utilisateurs)
Audit sécurité	OWASP ZAP	Analyse automatisée des vulnérabilités web (XSS, injection, exposition de tokens) en fin de sprint

Conclusion

Les besoins du système sont désormais définis et les choix technologiques validés. Le chapitre suivant entre dans la phase de conception, en détaillant le product backlog Scrum, l'architecture du système, le modèle de données et les machines à états.

Conception du système

Plan

Introduction	55
4.1 Product Backlog	55
4.1.1 Priorités MoSCoW	55
4.1.2 Story points	55
4.1.3 Tableau du Product Backlog	56
4.2 Planification des sprints	59
4.2.1 Diagramme de Gantt	60
4.3 Architecture logicielle	63
4.3.1 Vue d'ensemble	63
4.3.2 Couche présentation — Next.js 14	64
4.3.3 Couche applicatif — Django 5 + Django Ninja	64
4.3.4 Couche traitement asynchrone — Django-Q2	64
4.3.5 Couche données — PostgreSQL 16	64
4.3.6 Couche intégration — Adaptateurs ETL	65
4.3.6.1 Stratégie d'upsert	65
4.3.6.2 Clé de jointure : l'INE	66
4.3.6.3 Stratégie de synchronisation	66
4.3.7 Sécurité et contrôle d'accès	66
4.3.7.1 Flux d'authentification OIDC	66
4.3.7.2 Contrôle d'accès (RBAC)	66
4.3.7.3 Gestion du renouvellement des tokens JWT	67
4.3.7.4 Journal d'audit	67
4.3.7.5 Protection des données (RGPD)	68
4.4 Architecture physique et déploiement	68
4.4.1 Vue d'ensemble	68
4.4.2 Containers Docker	69
4.4.3 Flux réseau	69
4.4.4 Environnements	70
4.4.5 CI/CD	70
4.4.6 Stratégie de migration de schéma	70
4.4.7 Stratégie de rollback	70
4.4.8 Politique de sauvegarde	71
4.4.9 Gestion des secrets	71

4.5	Modèle de données	72
4.5.1	Organisation par packages	72
4.5.2	Principes structurants	72
4.5.2.1	L'année universitaire comme axe temporel.	72
4.5.2.2	L'INE comme clé de jointure universelle.	73
4.5.2.3	Séparation quotas INP / quotas N7.	73
4.6	Machines à états finis (FSM)	73
4.6.1	Principe et implémentation	73
4.6.2	FSM de l'année universitaire	73
4.6.3	FSM des entités métier	74
4.7	Flux de données : de la source au workflow	75
4.7.1	Détail de l'étape de nettoyage et validation	75
4.7.1.1	Description du processus de transformation	76
4.7.1.1.1	Gestion des sources multiples	76
4.7.1.1.2	Standardisation des formats	76
4.7.1.1.3	Normalisation sémantique	76
4.7.1.1.4	Gestion des valeurs manquantes	77
4.7.1.1.5	Détection et gestion des doublons	77
4.7.1.1.6	Validation métier	77
4.7.1.1.7	Traçabilité et gestion des erreurs	78
4.7.1.2	Architecture du pipeline de nettoyage	78
4.7.1.3	Technologies et outils utilisés	79
4.8	Gestion des risques	79
	Conclusion	80

Introduction

Ce chapitre présente la conception globale du système : le product backlog Scrum qui structure le développement, l'architecture logicielle et physique, le modèle de données et les machines à états qui gouvernent les cycles de vie métier.

4.1 Product Backlog

4.1.1 Priorités MoSCoW

Chaque user story est priorisée selon la méthode MoSCoW :

- M pour "Must Have" (Doit avoir) : Il s'agit des fonctionnalités qui sont absolument essentielles au succès du projet. Si ces fonctionnalités ne sont pas livrées, le projet est considéré comme un échec.
- S pour "Should Have" (Devrait avoir) : Il s'agit des fonctionnalités qui sont importantes mais pas vitales pour le succès du projet. Si possible, elles devraient être livrées, mais il est acceptable de les reporter à une version ultérieure si nécessaire.
- C pour "Could Have" (Pourrait avoir) : Il s'agit des fonctionnalités qui sont souhaitables mais pas essentielles pour le succès du projet. Elles ne sont intégrées à un sprint que si la capacité restante le permet, après réalisation et validation des fonctionnalités MVP prévues dans les sprints prioritaires.
- W pour "Won't Have" (N'aura pas) : Il s'agit des fonctionnalités qui ne seront pas livrées dans cette version du projet, mais qui peuvent être considérées pour une version ultérieure.

4.1.2 Story points

Les story points mesurent l'effort relatif selon l'échelle de Fibonacci.

SP	Effort estimé	Exemples typiques
1	Très faible — quelques heures	Ajout d'un champ, endpoint simple
2	Faible — une demi-journée	Nouveau modèle Django, endpoint CRUD
3	Modéré — une journée	Service métier, FSM à 3 états
5	Significatif — deux jours	Pipeline d'import, interface multi-composants
8	Élevé — trois à quatre jours	Intégration SSO, import Excel avec rapport
13	Très élevé — une semaine	Algorithme d'affectation, moteur de recommandation
21	Exceptionnel — plus d'une semaine	Chatbot RAG complet

TABLE 4.1 – Échelle des story points

4.1.3 Tableau du Product Backlog

TABLE 4.2 – Product Backlog

ID	Titre	User Story	Prior.	SP	Sprint
Fondations techniques					
US-01	Référentiels de base	En tant qu' administrateur RI , je veux disposer d'un référentiel stable des pays, départements et universités partenaires (synchronisé depuis MoveON) afin que toutes les entités du système s'appuient sur des données de référence cohérentes.	M	5	S1
US-02	Année universitaire et FSM	En tant qu' administrateur RI , je veux que le cycle de vie de l'année universitaire soit géré par une machine à états (initialisation → recommandation → consolidation → pré-affectation → validation → clôturée) afin de contrôler chaque phase de la campagne de manière sécurisée.	M	4	S1
US-03	Accords et quotas	En tant qu' administrateur RI , je veux un modèle de données complet pour les accords et leurs quotas (INP global, estimation N7, ajustement N7, répartition par département), synchronisé depuis MoveON, afin d'avoir une base fiable pour l'algorithme d'affectation.	M	8	S1
Données et traçabilité					
US-04	Audit log et traçabilité	En tant qu' administrateur RI , je veux que chaque action importante (import, transition FSM, modification, validation) soit enregistrée avec l'auteur et l'horodatage, afin de garantir la traçabilité complète des décisions.	M	3	S2
US-05	Données étudiants	En tant qu' administrateur RI , je veux importer et synchroniser les données académiques des étudiants (INE, GPA, département, niveau) depuis Pégase, afin d'alimenter l'algorithme d'affectation avec des données étudiants fiables et à jour.	M	8	S2
US-06	Vœux étudiants	En tant qu' administrateur RI , je veux importer les dossiers de candidature et les vœux ordonnés des étudiants depuis MoveON, et les consulter par étudiant, afin de les intégrer dans le système d'affectation.	M	5	S2

ID	Titre	User Story	Prior.	SP	Sprint
US-07	Rapport d'import	En tant qu' administrateur RI , je veux consulter un rapport détaillé après chaque import (succès, erreurs, doublons) avec la raison précise de chaque rejet, afin de corriger les anomalies rapidement.	M	3	S2
Algorithme d'affectation					
US-08	Gel du GPA	En tant qu' administrateur RI , je veux figer le GPA de chaque étudiant à la date de clôture définie afin de garantir la reproductibilité des résultats de l'algorithme.	M	3	S3
US-09	Algorithme Gale-Shapley	En tant qu' administrateur RI , je veux lancer l'algorithme d'affectation (Deferred Acceptance) qui affecte chaque étudiant à son meilleur vœu disponible en respectant les quotas par département, afin de produire une proposition équitable et stable.	M	13	S3
US-10	Supervision des affectations	En tant qu' administrateur RI , je veux consulter les propositions d'affectation, les ajuster manuellement pour les cas particuliers et tracer chaque modification avec un motif, avant de valider et publier les résultats.	M	8	S3
US-11	Consultation des résultats	En tant qu' étudiant , je veux consulter mon affectation une fois les résultats publiés afin de connaître ma destination et les démarches à suivre.	M	3	S3
Sources complémentaires et mobilités					
US-12	Stages internationaux (Eudonet)	En tant qu' administrateur RI , je veux importer et synchroniser les conventions de stages internationaux depuis Eudonet afin de les comptabiliser dans le calcul CTI de chaque étudiant.	M	8	S4
US-13	Mobilités entrantes (Excel)	En tant qu' administrateur RI , je veux importer les mobilités entrantes via un template Excel standardisé afin de centraliser le suivi des étudiants internationaux accueillis à l'ENSEEIH.	M	8	S4
US-14	Réconciliation sans INE	En tant qu' administrateur RI , je veux que le système me propose des correspondances pour les enregistrements sans INE valide (via nom, prénom, date de naissance) afin de résoudre manuellement les cas ambigus.	S	5	S4

ID	Titre	User Story	Prior.	SP	Sprint
US-15	Déclaration mobilité complémentaire	En tant qu' étudiant , je veux déclarer une expérience internationale hors cursus (summer school, séjour linguistique, programme court) avec dépôt de justificatifs, afin qu'elle soit prise en compte dans mon calcul de durée CTI.	M	8	S4
US-16	Validation des mobilités complémentaires	En tant qu' administrateur RI , je veux consulter les justificatifs soumis, valider ou rejeter chaque demande avec un motif, afin de maintenir la qualité des données comptabilisées dans le calcul CTI.	M	5	S4
Conformité CTI					
US-17	Calcul automatique des durées CTI	En tant qu' administrateur RI , je veux que le système calcule automatiquement la durée totale de mobilité de chaque étudiant (séjours académiques, stages, expériences complémentaires validées) afin de vérifier la conformité CTI sans intervention manuelle.	M	8	S5
US-18	Indicateur CTI pour l'étudiant	En tant qu' étudiant , je veux visualiser ma durée cumulée de mobilité et sa comparaison avec le seuil CTI requis afin de savoir si je suis en règle pour l'obtention de mon diplôme.	M	3	S5
US-19	Export rapport CTI	En tant qu' administrateur RI , je veux exporter un rapport de conformité CTI pour l'ensemble des étudiants de la promotion afin de le soumettre lors des audits.	S	5	S5
Espace étudiant					
US-20	Catalogue des destinations	En tant qu' étudiant , je veux consulter les universités partenaires avec les places disponibles estimées, les conditions d'éligibilité (GPA minimum, département) et les informations pratiques, afin d'explorer les options avant de saisir mes vœux dans MoveON.	M	5	S5
US-21	Suivi de candidature	En tant qu' étudiant , je veux suivre le statut de ma candidature en temps réel (soumis, en traitement, affecté, rejeté) afin d'être informé de l'avancement sans solliciter le service RI.	M	3	S5
US-22	Gestion des mobilités entrantes	En tant qu' administrateur RI , je veux gérer le suivi administratif des étudiants internationaux accueillis (procédure envoyée, candidature Moodle, lettre d'acceptation, inscription) afin de centraliser le suivi dans un seul outil.	M	5	S5

ID	Titre	User Story	Prior.	SP	Sprint
Supervision et authentification					
US-23	Console de supervision de campagne	En tant qu' administrateur RI , je veux un tableau de bord affichant l'état en temps réel de la campagne (candidatures reçues, imports réalisés, anomalies en attente, avancement par phase) afin de piloter efficacement la campagne.	S	8	S5
US-24	Authentification SSO	En tant qu' utilisateur , je veux m'authentifier via le SSO institutionnel (CAS/OIDC) afin d'accéder à mon espace personnalisé (étudiant ou administrateur) sans créer de compte séparé.	M	8	S5
US-25	Tableaux de bord analytiques et exports CTI	En tant qu' administrateur RI , je veux disposer de tableaux de bord analytiques et d'exports CTI (taux d'affectation par département, évolution des flux entrants/sortants, destinations les plus demandées, indicateurs CTI) afin d'alimenter le pilotage opérationnel et la réflexion stratégique sur les partenariats.	M	8	S5

4.2 Planification des sprints

Le développement est organisé en cinq sprints MVP de deux à trois semaines. Les fonctionnalités classées *Could Have* restent hors engagement initial : elles ne pourront être ajoutées à une itération ultérieure que si les sprints MVP sont réalisés et validés dans la capacité prévue.

TABLE 4.3 – Planification des sprints

Sprint	Thème	User stories couvertes
Sprint 1	Fondations techniques	US-01 à US-03 — Référentiels de base (sync MoveON), année universitaire et FSM, accords et quotas (sync MoveON)
Sprint 2	Données et traçabilité	US-04 à US-07 — Audit log, données étudiants (import Pégase), vœux étudiants (import MoveON), rapport d'import
Sprint 3	Algorithme d'affectation	US-08 à US-11 — Gel du GPA, algorithme Gale-Shapley, supervision et publication des résultats, consultation des résultats par l'étudiant

Sprint	Thème	User stories couvertes
Sprint 4	Sources complémentaires et mobilités	US-12 à US-16 — Stages internationaux (import Eudonet), mobilités entrantes (import Excel), réconciliation sans INE, déclaration et validation des mobilités complémentaires
Sprint 5	CTI, espace étudiant et supervision	US-17 à US-25 — Calcul des durées CTI, indicateur étudiant, export CTI, catalogue des destinations, suivi de candidature, gestion mobilités entrantes, console de supervision, SSO, tableaux de bord analytiques et exports CTI

4.2.1 Diagramme de Gantt

TABLE 4.4 – Diagramme de Gantt

Activité	Mois 1				Mois 2				Mois 3				Mois 4				Mois 5				Mois 6			
	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24
Phase 1 — Cadrage et analyse																								
Analyse des besoins																								
Rédaction CDC / spécifications																								
Modélisation des données																								
Conception architecture																								
Phase 2 — Environnement et DevOps																								
Setup Docker / environnement																								
Pipeline CI/CD (Git-Lab/GitHub)																								
Configuration PostgreSQL																								
Setup Django-Q2 / MinIO																								
Monitoring (Sentry / Grafana)																								
Phase 3 — Développement itératif																								
Sprint 1																								
Sprint 2																								
Sprint 3																								
Sprint 4																								

Activité	Mois 1				Mois 2				Mois 3				Mois 4				Mois 5				Mois 6			
	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24
Sprint 5																								
Phase 4 — Tests et qualité																								
Tests unitaires (continus)																								
Tests d'intégration (continus)																								
Tests de charge (Locust)																								
Audit sécurité (OWASP ZAP)																								
UAT avec service RI																								
Phase 5 — Livraison																								
Documentation technique																								
Déploiement staging																								
Livraison finale																								

4.3 Architecture logicielle

4.3.1 Vue d'ensemble

L'architecture suit un modèle **3-tiers** classique, enrichi d'une couche de traitement asynchrone. Les trois niveaux sont clairement séparés : présentation (Next.js), applicatif (Django + Django Ninja) et données (PostgreSQL). Les traitements lourds sont délégués à une couche asynchrone indépendante (Django-Q2).

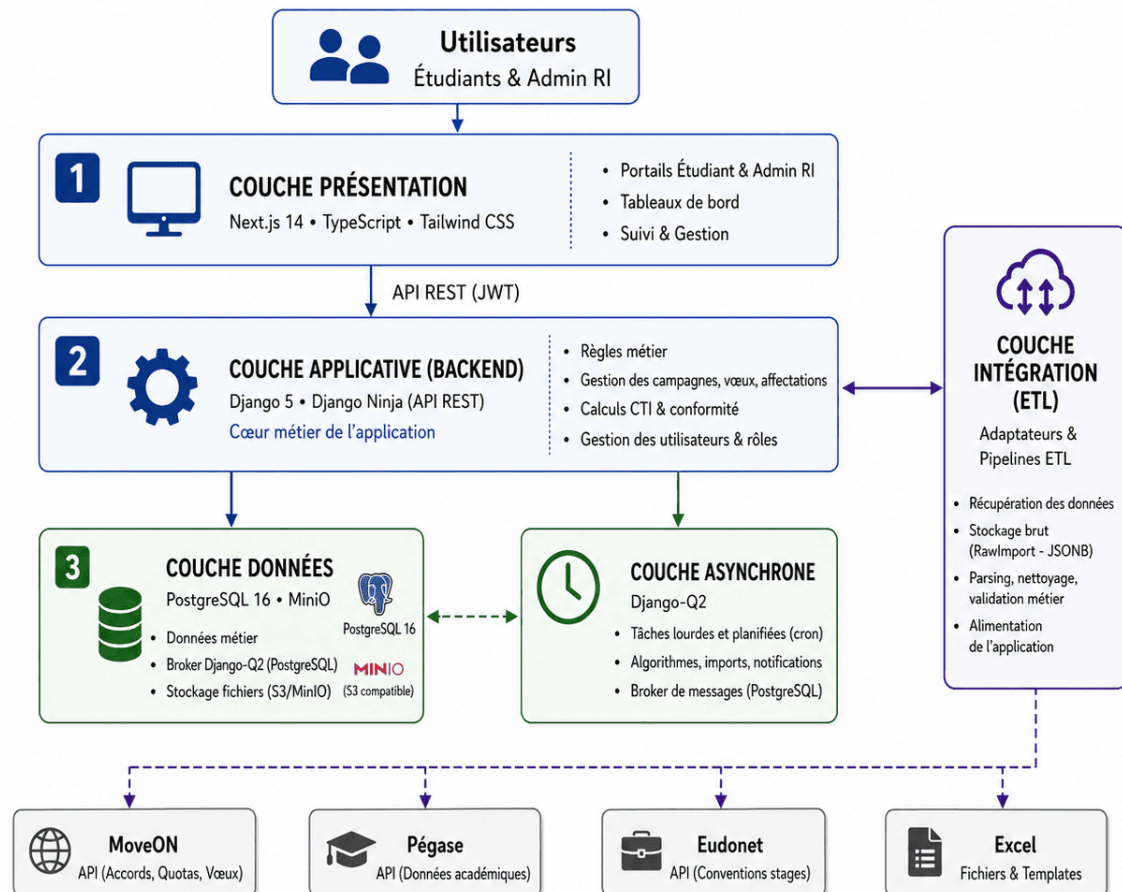


FIGURE 4.1 – Architecture logicielle

La figure 4.1 représente l'architecture logicielle du système selon un modèle en cinq couches. La couche **Présentation** (Next.js 14) communique avec la couche **Applicative** (Django 5 + Django Ninja) exclusivement via des appels REST/JSON sur HTTPS ; aucun composant Next.js n'accède directement à la base de données. La couche **Asynchrone** (Django-Q2) partage la même base de données que la couche applicative — PostgreSQL joue ici le rôle de broker de messages — et s'exécute dans un processus distinct pour ne pas bloquer les requêtes HTTP. La couche **Données** regroupe PostgreSQL pour la persistance relationnelle et JSONB, et MinIO pour les fichiers binaires ; ces deux services sont internes et non accessibles directement depuis l'extérieur. La couche **Intégration** rassemble les adaptateurs ETL : elle est positionnée à part des couches métier car elle dépend de systèmes

externes (MoveON, Pégase, Eudonet) dont la disponibilité n'est pas garantie. L'authentification suit un flux OIDC : l'utilisateur est redirigé vers le CAS de Toulouse INP, qui renvoie un token JWT ; la couche applicative valide ce token à chaque requête et en extrait le rôle (ADMIN_RI ou ETUDIANT).

4.3.2 Couche présentation — Next.js 14

Le frontend est développé avec **Next.js 14**, **TypeScript** et **Tailwind CSS**. Il expose deux espaces distincts selon le rôle de l'utilisateur authentifié.

Espace administrateur RI : console de supervision de campagne, gestion des quotas, supervision des affectations, instruction des mobilités complémentaires, tableaux de bord CTI.

Espace étudiant : catalogue des destinations, suivi de candidature, déclaration d'expériences complémentaires, indicateur CTI.

Le frontend communique exclusivement avec le backend via l'API REST au format JSON, avec authentification par token JWT.

4.3.3 Couche applicatif — Django 5 + Django Ninja

Le backend est le cœur du système. Il est découpé en modules Django autonomes, chacun responsable d'un domaine métier.

Django Ninja est utilisé pour exposer l'API REST. Chaque module définit ses propres schémas pour valider les données en entrée et structurer les réponses. L'authentification repose sur le middleware OIDC (CAS Toulouse INP), qui vérifie le token et associe l'utilisateur à chaque requête.

4.3.4 Couche traitement asynchrone — Django-Q2

Les traitements qui ne doivent pas bloquer l'interface sont délégués à **Django-Q2**, qui utilise PostgreSQL directement comme broker de messages. Deux catégories de tâches sont gérées :

- **Tâches déclenchées par événement** : lancement de l'algorithme d'affectation, recalcul des durées CTI après chaque transition FSM, génération des rapports d'import.
- **Tâches planifiées (cron)** : synchronisation nocturne avec MoveON, Pégase et Eudonet ; purge des données brutes (**RawImport**) traitées depuis plus de 90 jours.

4.3.5 Couche données — PostgreSQL 16

PostgreSQL 16 est le seul système de persistance du projet. Il joue trois rôles distincts :

- **Base de données relationnelle** : toutes les entités métier y sont stockées, avec des index composites sur `academic_year_id` pour les performances de filtrage par campagne.
- **Broker de messages Django-Q2** : la table `django_q_task` stocke les tâches en attente.
- **Stockage des données brutes (JSONB)** : la table `RawImport` conserve les réponses JSON brutes de chaque appel API avant transformation, garantissant la possibilité de relancer les imports.

Le stockage des fichiers (justificatifs de mobilités complémentaires, templates Excel) est assuré par **MinIO**, un serveur de stockage objet compatible S3, déployé comme container Docker interne.

4.3.6 Couche intégration — Adaptateurs ETL

Chaque source externe dispose d'un module adaptateur isolé :

- `adapters/moveon.py` : extraction des accords, quotas et vœux ;
- `adapters/pegase.py` : extraction des données académiques (INE, GPA, département) ;
- `adapters/eudonet.py` : extraction des conventions de stages internationaux ;
- `adapters/excel.py` : parsing des templates Excel standardisés (mobilités entrantes, fallbacks).

Chaque adaptateur suit le pipeline ETL suivant : récupération → stockage brut (JSONB) → parsing → standardisation → validation métier → persistance → rapport d'import.

En cas d'échec API, un mécanisme de retry exponentiel (3 tentatives) est appliqué avant de déclencher une alerte via **Sentry**.

4.3.6.1 Stratégie d'upsert

Lors de la phase de persistance, chaque enregistrement issu du pipeline est traité selon une logique d'upsert : si un enregistrement correspondant n'existe pas encore en base, il est inséré ; s'il existe déjà (identifié par la clé métier, en général l'INE ou l'identifiant de l'accord), il est comparé champ par champ à la version en base. Si les valeurs sont identiques, l'enregistrement est ignoré. Si des différences sont détectées, la mise à jour est appliquée et la modification est tracée dans le journal d'audit — ce qui permet au service RI de savoir précisément quelles données ont changé d'une synchronisation à l'autre (par exemple, un GPA révisé dans Pégase ou une date de fin de stage corrigée dans Eudonet).

4.3.6.2 Clé de jointure : l'INE

L'identifiant national étudiant (INE) est la clé partagée entre MoveON, Pégase, Eudonet et le CAS institutionnel. Toutes les données importées sont croisées via l'INE pour construire une vue unifiée du dossier étudiant.

Lorsque l'INE est absent d'une source (notamment Eudonet), une stratégie de réconciliation est appliquée : matching déterministe sur le triplet nom/prénom/date de naissance, avec détection des cas ambigus remontés pour validation manuelle par le service RI.

4.3.6.3 Stratégie de synchronisation

Source	Déclencheur	Remarque
MoveON	Tâche Django-Q2 planifiée (phases actives) + manuelle avant lancement de l'algorithme	Lecture seule — aucune écriture vers MoveON
Pégase	Initialisation de campagne + à la demande	GPA figé à la <code>gpa_freeze_date</code> définie par l'admin
Eudonet	Périodique (fin de période de stage) + à la demande	Réconciliation INE si absent
Excel	Upload manuel par l'administrateur	Fallback pour toutes les sources en cas d'indisponibilité API

TABLE 4.5 – Stratégie de synchronisation par source

4.3.7 Sécurité et contrôle d'accès

4.3.7.1 Flux d'authentification OIDC

L'authentification est assurée par le CAS institutionnel de Toulouse INP via le protocole OIDC. Le flux se déroule en quatre étapes :

1. L'utilisateur accède à l'application et est redirigé vers le portail SSO institutionnel.
2. Après authentification CAS réussie, un token OIDC (JWT) est émis et transmis à l'application.
3. Le backend décode le token, récupère l'INE et le groupe LDAP de l'utilisateur, et détermine automatiquement son rôle (`ADMIN_RI` ou `ETUDIANT`).
4. Le frontend reçoit le rôle et affiche l'espace correspondant. Les tokens JWT ont une durée de vie courte (15 minutes), avec un mécanisme de renouvellement géré côté serveur.

4.3.7.2 Contrôle d'accès (RBAC)

Deux rôles sont définis et détectés automatiquement depuis l'annuaire institutionnel :

- **ADMIN_RI** : accès complet aux fonctionnalités d’administration — imports, algorithme, validations, exports CTI. Toute action est tracée dans l’AuditLog.
- **ETUDIANT** : accès restreint à son propre dossier uniquement. Un contrôle de type *object-level permission* est appliqué sur chaque endpoint exposant des données personnelles.

4.3.7.3 Gestion du renouvellement des tokens JWT

Le token JWT émis par le CAS de Toulouse INP a une durée de vie courte (15 minutes) pour limiter le risque en cas d’interception. Côté Next.js, NextAuth gère automatiquement le renouvellement : lorsque le token approche de son expiration, NextAuth initie silencieusement un flux de rafraîchissement via le endpoint OIDC du CAS sans interrompre la navigation de l’utilisateur. Côté backend Django, chaque requête API passe par un middleware qui vérifie la signature et l’expiration du token ; si le token est expiré, la requête est rejetée avec un code 401 et le frontend déclenche un nouveau cycle d’authentification. Cette architecture garantit qu’un token révoqué au niveau du CAS ne peut plus être utilisé pour accéder au backend, même si le frontend n’a pas encore détecté l’expiration.

4.3.7.4 Journal d’audit

Le journal d’audit est géré par `django-auditlog`, configuré pour enregistrer automatiquement les événements suivants :

- Authentifications réussies et tentatives échouées ;
- Toutes les transitions FSM (changements d’état d’une année universitaire, d’un dossier, d’une affectation) ;
- Modifications manuelles des affectations par le service RI (avec champ *motif* obligatoire) ;
- Lancements d’imports ETL et résultats (source, nombre d’enregistrements traités, erreurs) ;
- Validations ou rejets de mobilités complémentaires ;
- Exports de rapports CTI.

Chaque entrée enregistre l’identifiant de l’utilisateur auteur de l’action, l’horodatage UTC, l’entité concernée (table + clé primaire), l’action réalisée et les valeurs avant/après modification. Le journal est stocké dans une table PostgreSQL dédiée (`auditlog_logentry`), accessible en lecture seule depuis l’interface d’administration Django.

La politique de rétention est de 7 ans, alignée sur les exigences de traçabilité. La consultation est possible depuis l’interface d’administration Django (filtres par utilisateur, par entité, par période) ou via un endpoint API réservé au rôle **ADMIN_RI**. Aucune entrée d’audit ne peut être modifiée ou supprimée depuis l’application — les suppressions éventuelles nécessitent un accès direct à la base de données avec une traçabilité opérateur.

4.3.7.5 Protection des données (RGPD)

- **Chiffrement au repos** : les champs sensibles (INE, email, date de naissance) sont chiffrés en base via `pgcrypto`.
- **Chiffrement en transit** : TLS obligatoire sur tous les endpoints.
- **Stockage des justificatifs** : fichiers chiffrés (SSE-S3) dans MinIO, supprimés automatiquement après validation et expiration du délai de conservation.
- **Droit à l'oubli** : procédure d'anonymisation des données personnelles après diplôme, tout en conservant les statistiques agrégées pour les rapports CTI.

4.4 Architecture physique et déploiement

4.4.1 Vue d'ensemble

L'application est déployée sur un serveur on-premise de Toulouse INP. L'ensemble des services est conteneurisé via **Docker Compose**, ce qui garantit la portabilité entre environnements et simplifie le déploiement.

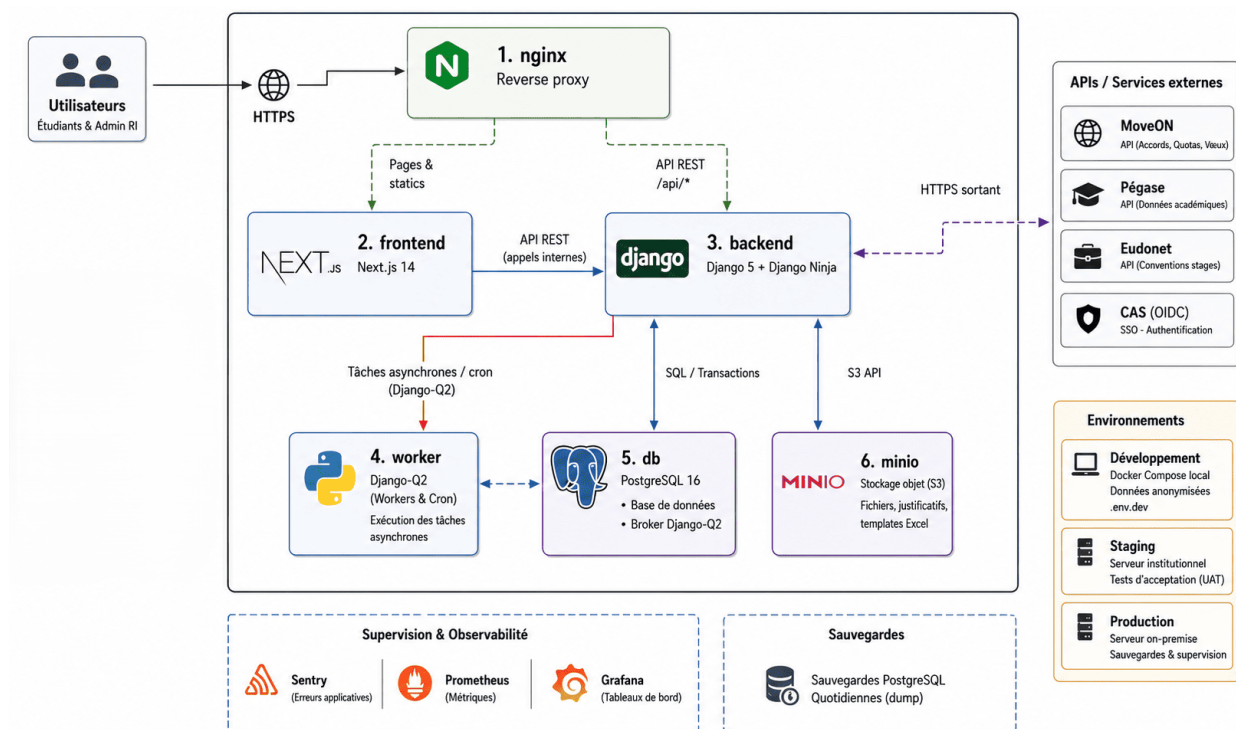


FIGURE 4.2 – Architecture physique

La figure 4.2 représente le déploiement physique de l'application sur le serveur on-premise de Toulouse INP. L'ensemble des services s'exécute dans un réseau Docker interne (**bridge**) ; seul le conteneur **Nginx** est accessible depuis l'extérieur, sur les ports 80 (HTTP, redirigé vers HTTPS) et 443 (HTTPS). En interne, Nginx route vers `frontend:3000` pour les pages

Next.js et vers `backend:8000` pour les appels API. Les containers `db` (PostgreSQL, port 5432), `minio` (port 9000) et `worker` (Django-Q2) sont isolés du réseau public — ils ne reçoivent aucune connexion entrante directe. **Sentry** est un service externe hébergé en dehors de l’infrastructure : le backend et le frontend lui envoient des rapports d’erreurs via HTTPS sortant. **Prometheus** tourne comme container interne pour la collecte de métriques, et **Grafana** (également interne, accessible uniquement depuis le réseau institutionnel) pour leur visualisation. Les appels aux APIs externes — MoveON, Pégase, Eudonet, CAS — sont initiés depuis le container `backend` uniquement, via HTTPS sortant depuis le réseau du serveur.

4.4.2 Containers Docker

L’infrastructure est composée de six containers :

Container	Image	Rôle
<code>nginx</code>	<code>nginx:alpine</code>	Reverse proxy : SSL/TLS, fichiers statiques, routage vers Django et Next.js
<code>frontend</code>	<code>node:20-alpine</code>	Application Next.js (SSR + API routes)
<code>backend</code>	<code>python:3.12-slim</code>	Django 5 + Django Ninja
<code>worker</code>	<code>python:3.12-slim</code>	Django-Q2 cluster (tâches asynchrones et cron)
<code>db</code>	<code>postgres:16-alpine</code>	PostgreSQL — base de données et broker Django-Q2
<code>minio</code>	<code>minio/minio</code>	Stockage objet S3 (justificatifs, fichiers Excel)

TABLE 4.6 – Containers Docker et leurs rôles

4.4.3 Flux réseau

Les requêtes entrantes arrivent sur **Nginx**. Nginx route vers :

- le container `frontend` pour les pages et ressources statiques Next.js ;
- le container `backend` pour les appels à l’API REST (`/api/*`).

Le container `backend` communique avec `db` pour toutes les opérations de données, et avec `minio` pour le stockage de fichiers. Le container `worker` partage la même base de données que `backend` — c’est ainsi que Django-Q2 utilise PostgreSQL comme broker sans service supplémentaire. Les APIs externes (MoveON, Pégase, Eudonet, CAS) sont accessibles depuis le serveur via HTTPS sortant.

4.4.4 Environnements

Trois environnements sont maintenus :

- **Développement** : Docker Compose local avec des jeux de données anonymisés. Les variables d'environnement sont chargées depuis un fichier `.env`.
- **Staging** : reproduction fidèle de la production sur le serveur institutionnel, utilisée pour les tests d'acceptation (UAT) avec le service RI avant chaque mise en production.
- **Production** : déploiement final sur le serveur on-premise, avec sauvegardes PostgreSQL quotidiennes et supervision via **Sentry** (erreurs applicatives) et **Prometheus/Grafana** (métriques système).

4.4.5 CI/CD

Un pipeline CI/CD est mis en place sur **GitHub Actions** avec trois étapes automatisées à chaque push :

- **Lint + tests unitaires** : exécution de la suite de tests Python (pytest) et du linter (ruff) ;
- **Build** : construction des images Docker ;
- **Déploiement automatique** sur staging à chaque merge sur la branche `main`.

4.4.6 Stratégie de migration de schéma

Django gère les évolutions du schéma de base de données via son système de migrations intégré. À chaque déploiement, les migrations sont appliquées avant le démarrage du serveur applicatif afin de garantir la cohérence entre le code exécuté et la structure de la base de données.

La stratégie retenue favorise les migrations additives en deux temps : on commence par ajouter le nouveau champ ou la nouvelle table (migration non destructive), puis on supprime les éléments obsolètes dans une migration ultérieure une fois que le code de production ne les référence plus. Cette approche évite les ruptures entre le code et le schéma lors d'un déploiement sans fenêtre de maintenance.

Les migrations Django sont versionnées dans le dépôt git, au même titre que le code. Toute modification du modèle donne lieu à un fichier de migration dédié, relu avant intégration afin de vérifier qu'il respecte la stratégie de déploiement retenue.

4.4.7 Stratégie de rollback

En cas d'incident après un déploiement, le rollback se déroule en deux étapes. D'abord, l'image Docker précédente est remise en service via Docker Compose en pointant vers le tag de la version antérieure. Ensuite, si la migration de schéma était irréversible (ex. : suppression

de colonne), une restauration à partir de la sauvegarde pré-déploiement est effectuée. Pour cette raison, une sauvegarde complète de la base de données est systématiquement déclenchée avant chaque déploiement en production.

Lorsque les migrations sont réversibles, le retour à une version antérieure du schéma peut être réalisé sans restauration complète. L'état des migrations est contrôlé avant toute opération afin d'identifier précisément la version cible et d'éviter les incohérences entre l'application et la base de données.

4.4.8 Politique de sauvegarde

Deux niveaux de sauvegarde coexistent.

Sauvegardes logiques quotidiennes : une sauvegarde logique de la base est générée chaque nuit dans un format restaurable PostgreSQL. Ces sauvegardes sont conservées pendant 30 jours et stockées sur un volume séparé du serveur, distinct du volume de données PostgreSQL.

Sauvegardes physiques hebdomadaires : une copie complète des données PostgreSQL est réalisée chaque semaine. Elle permet une restauration plus fine en combinaison avec les journaux transactionnels archivés. Ces sauvegardes sont conservées pendant 6 mois.

Un test de restauration est effectué manuellement une fois par mois sur l'environnement staging : la procédure consiste à restaurer la sauvegarde la plus récente et à vérifier l'intégrité des données via un script de contrôle qui vérifie les comptages par table et les contraintes référentielles. Ce test documente le *Recovery Time Objective* (RTO) réel et détecte d'éventuels problèmes avant qu'ils ne surviennent en production.

4.4.9 Gestion des secrets

Aucun secret n'est présent dans le dépôt git. Les variables sensibles — `SECRET_KEY` Django, mots de passe PostgreSQL, clés d'API MoveON/Pégase/Eudonet, tokens MinIO — sont injectées au runtime via des variables d'environnement déclarées dans un fichier `.env.prod` hébergé uniquement sur le serveur de production, et jamais versionné. Le fichier `.gitignore` exclut explicitement tous les fichiers `.env.*` à l'exception du fichier de développement avec des valeurs fictives.

En développement local, un fichier de configuration contenant uniquement des valeurs fictives ou non sensibles est fourni à titre de référence. La séparation claire entre les environnements garantit qu'aucune valeur de production ne peut se retrouver accidentellement dans le code source.

4.5 Modèle de données

Le modèle de données est structuré en huit packages cohérents, chacun regroupant les entités d'un même domaine métier. Une vue d'ensemble des entités et de leurs relations est présentée ci-dessous ; les classes impliquées dans chaque sprint seront détaillées dans des diagrammes de classes dédiés.

4.5.1 Organisation par packages

Le modèle s'articule autour de huit packages dont les responsabilités sont résumées dans le tableau suivant.

Package	Responsabilité
Reference	Référentiels stables partagés par l'ensemble du système : pays, régions CTI, départements N7.
Academic Year	Axe temporel central. Chaque campagne de mobilité est rattachée à une année universitaire dont le cycle de vie est piloté par une machine à états.
Students	Données académiques des étudiants importées depuis Pégase : Student , inscriptions annuelles (AnnualEnrollment), parcours pédagogique (Parcours), relevés de GPA gelés à date fixe.
Institutions	Universités partenaires (synchronisées depuis MoveON) et organismes d'accueil pour les stages.
Agreements & Quotas	Accords de mobilité (Agreement) et leur hiérarchie de quotas : déclinaison annuelle par accord (AgreementYear), liste des départements autorisés par accord (AgreementDepartment), et répartition annuelle du quota N7 par département (AgreementYearDepartment).
Applications	Dossiers de candidature, vœux ordonnés et affectations produites par l'algorithme ou saisies manuellement.
Mobilities	Quatre types de mobilités : sortantes académiques, stages internationaux (Eudonet), complémentaires déclarées par l'étudiant, et entrantes (import Excel). La durée CTI consolidée est calculée automatiquement.
Import	Stockage brut (JSONB) des données reçues de chaque source avant transformation, garantissant la reprise sur erreur et la traçabilité des imports.

TABLE 4.7 – Packages du modèle de données

4.5.2 Principes structurants

Trois principes d'architecture transversaux gouvernent l'ensemble du modèle.

4.5.2.1 L'année universitaire comme axe temporel.

Toutes les entités opérationnelles (dossiers, vœux, affectations, mobilités, imports) sont rattachées à une **AcademicYear**. Ce rattachement garantit l'historisation complète des campagnes passées et le basculement automatique en lecture seule dès la clôture d'une année.

4.5.2.2 L'INE comme clé de jointure universelle.

L'identifiant national étudiant (INE) est la clé partagée entre toutes les sources externes — MoveON, Pégase, Eudonet et le CAS institutionnel. Lorsqu'un enregistrement arrive sans INE valide (cas fréquent dans Eudonet), une stratégie de réconciliation déterministe est appliquée sur le triplet nom / prénom / date de naissance, avec remontée manuelle des cas ambigus.

4.5.2.3 Séparation quotas INP / quotas N7.

Les accords MoveON sont définis à l'échelle de l'INP global. Le modèle intègre une hiérarchie de quotas à quatre niveaux — quota INP importé, estimation N7 calculée, ajustement manuel de l'administrateur, et répartition par département N7 — afin de respecter les contraintes institutionnelles tout en offrant la granularité nécessaire à l'algorithme d'affectation.

4.6 Machines à états finis (FSM)

4.6.1 Principe et implémentation

Plusieurs entités du système voient leur cycle de vie contrôlé par une machine à états finis, implémentée via `django-fsm`. Le champ de statut est déclaré avec l'option `protected=True` : toute tentative de modifier l'état directement — sans passer par une méthode de transition explicite — lève une exception. C'est le mécanisme qui garantit qu'une campagne clôturée ne peut pas être rouverte, qu'une affectation validée ne peut pas être modifiée silencieusement, et qu'aucune donnée non validée ne peut déclencher un workflow métier.

4.6.2 FSM de l'année universitaire

C'est la machine à états centrale du système. Elle gouverne l'accès à toutes les fonctionnalités de la campagne.

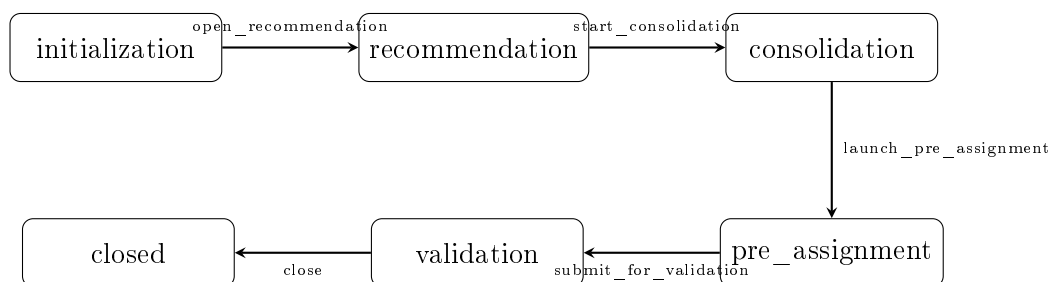


FIGURE 4.3 – Machine à états — AcademicYear

- **initialization** → **recommendation** (`open_recommendation`) : ouverture de la consultation des destinations aux étudiants.

- **recommendation** → **consolidation** (`start_consolidation`) : clôture des vœux, gel du GPA, déclenchement de la synchronisation finale avec MoveON.
- **consolidation** → **pre_assignment** (`launch_pre_assignment`) : lancement de l'algorithme Gale-Shapley.
- **pre_assignment** → **validation** (`submit_for_validation`) : soumission des propositions d'affectation à la validation du service RI.
- **validation** → **closed** (`close`) : publication des résultats, mise en lecture seule de toutes les données de la campagne.

Toutes les transitions sont déclenchables uniquement par un utilisateur avec le rôle `ADMIN_RI`. Le tableau suivant précise les conditions préalables vérifiées par le système avant d'autoriser chaque transition et le comportement en cas d'échec.

Transition	Préconditions (guard conditions)	Comportement en cas d'échec
<code>open_recommendation</code>	Accords importés depuis MoveON, quotas par département configurés	Exception levée, message d'erreur précis renvoyé au frontend
<code>start_consolidation</code>	Au moins un vœu saisi par les étudiants, date limite de saisie atteinte ou dépassée	Exception levée si aucun vœu importé
<code>launch_pre_assignment</code>	GPA gelé (synchronisation Pégase terminée), vœux importés depuis MoveON	Exception levée, tâche Django-Q2 non déclenchée
<code>submit_for_validation</code>	Algorithme exécuté sans erreur, toutes les affectations en état proposed	Transition bloquée si des affectations sont en état intermédiaire
<code>close</code>	Validation manuelle explicite du service RI sur les propositions	Transition refusée tant que des affectations restent non validées

TABLE 4.8 – Conditions préalables (guard conditions) des transitions FSM

4.6.3 FSM des entités métier

Entité	États et transitions
ApplicationFile	<code>open</code> → <code>submitted</code> → <code>processing</code> → <code>closed</code>
StudentWish	<code>candidate</code> → <code>selected</code> → <code>assigned</code> <code>rejected</code>
Assignment	<code>proposed</code> → <code>validated</code> → <code>published</code> <code>cancelled</code>
OutgoingMobility	<code>planned</code> → <code>ongoing</code> → <code>completed</code> <code>cancelled</code>
ComplementaryMobility	<code>submitted</code> → <code>under_review</code> → <code>validated</code> <code>rejected</code>
IncomingMobility	<code>nominated</code> → <code>imported</code> → <code>registered</code> → <code>active</code>

TABLE 4.9 – FSM des entités métier

Chaque transition sur **OutgoingMobility**, **InternationalInternship** ou **ComplementaryMobility** déclenche automatiquement une tâche Django-Q2 de recalcul de **MobilityDuration** pour l'étudiant concerné, garantissant que l'indicateur CTI est toujours à jour.

4.7 Flux de données : de la source au workflow

Le flux de données décrit le parcours d'une information depuis son import depuis MoveON jusqu'à son intégration dans les entités métier et les machines à états du système. Ce processus garantit une séparation claire entre la couche *transport* (récupération et préparation des données) et la couche *métier* (validation, persistance et déclenchement des workflows).

1. **Récupération (Transport)** : la tâche Django-Q2 interroge l'API MoveON via OAuth2 et récupère les vœux au format JSON. La réponse brute est stockée temporairement en JSONB dans PostgreSQL, garantissant la traçabilité et la reprise sur erreur.
2. **Nettoyage et validation (Python)** : un pipeline de transformation vérifie les formats, détecte les doublons et croise les INE avec la base Pégase. Les enregistrements invalides sont tracés dans un rapport d'import avec la raison du rejet.
3. **Persistance (PostgreSQL)** : les données validées sont insérées dans les tables métier (*SouhaitSejour*, *Accord*, etc.) et rattachées à l'année universitaire courante.
4. **Activation du workflow (FSM)** : chaque enregistrement créé déclenche la machine à états correspondante. Par exemple, un *SouhaitSejour* importé est initialisé au statut *Candidat*, et ne peut progresser vers *Affecté* que via les transitions autorisées.
5. **Supervision (Interface admin)** : l'administrateur consulte les résultats de l'import, résout les anomalies signalées et confirme le passage à la phase suivante de la campagne.

Ce découpage garantit qu'aucune donnée non validée ne peut déclencher un workflow métier, et que chaque étape est auditable de manière indépendante.

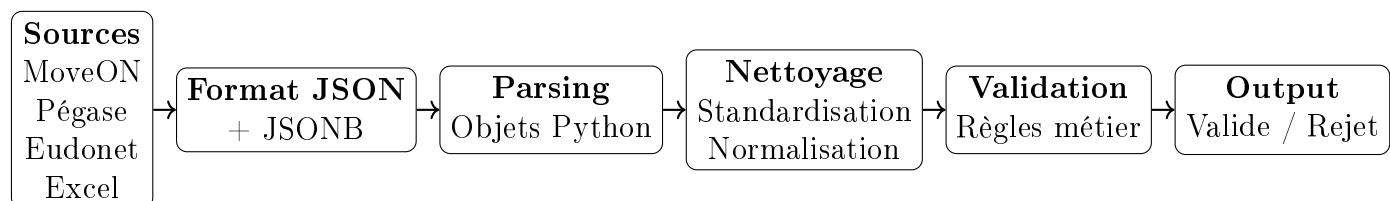


FIGURE 4.4 – Pipeline global de traitement des données

4.7.1 Détail de l'étape de nettoyage et validation

Dans le flux global de traitement des données, l'étape de **nettoyage et validation** constitue un sous-processus central garantissant la qualité et la cohérence des informations avant leur intégration dans le système métier.

Cette étape s’inscrit directement après la phase de récupération et de structuration des données (JSON/JSONB), et avant leur persistance en base de données.

Afin de clarifier son fonctionnement interne, nous détaillons ci-dessous le pipeline de traitement appliqué à cette phase.

4.7.1.1 Description du processus de transformation

Le nettoyage des données constitue une étape critique du système, garantissant la qualité, la cohérence et l’exploitabilité des informations avant leur intégration métier. Les données sont issues de plusieurs sources hétérogènes (MoveON, Pégase, Eudonet, ainsi que des fichiers Excel), nécessitant un processus de nettoyage et d’harmonisation. Ce pipeline suit une approche modulaire et déterministe, inspirée des principes ETL (*Extract - Transform - Load*).

4.7.1.1.1 Gestion des sources multiples

- Les données proviennent de sources hétérogènes présentant des structures et formats différents (API REST, bases de données, fichiers Excel).
- Une étape de transformation initiale permet de convertir ces données dans un format intermédiaire commun (JSON), facilitant leur traitement uniforme dans le pipeline.
- Cette homogénéisation permet d’appliquer les mêmes règles de nettoyage, de validation et de contrôle indépendamment de la source d’origine.

4.7.1.1.2 Standardisation des formats

- Normalisation des chaînes de caractères (suppression des espaces superflus, homogénéisation de la casse).
- **Gestion des accents :**
 - Les accents sont conservés pour les champs descriptifs (noms propres, universités, pays) afin de préserver l’intégrité sémantique des données.
 - Lors des opérations de recherche, de comparaison ou de détection de similarité (*matching*), les comparaisons sont effectuées de manière insensible aux accents afin d’éviter les divergences liées aux variations d’écriture.
- Uniformisation des formats de dates (ISO 8601).
- Conversion des types (ex : chaînes vers entiers pour les identifiants).

4.7.1.1.3 Normalisation sémantique

- Harmonisation des noms d’universités via une table de correspondance (*lookup table*) :

- Les données sources peuvent contenir différentes variantes pour une même université (abréviations, différences de langue, fautes de frappe).
- Une table de correspondance permet d’associer ces variantes à une valeur unique de référence.
- Cette approche garantit la cohérence des données et évite les duplications logiques lors des traitements.
- Normalisation des pays selon un référentiel standard (ISO 3166).
- Mapping des valeurs hétérogènes vers des valeurs métier uniques (ex : “FR”, “France”, “france” → “France”).

4.7.1.1.4 Gestion des valeurs manquantes

- Identification des champs obligatoires (INE, université, type de mobilité).
- Si une valeur obligatoire est manquante, l’enregistrement est rejeté et consigné dans le rapport d’import avec la raison de l’erreur.
- Si les champs manquants ne sont pas critiques, l’enregistrement est conservé et peut être sauvegardé en base de données.

4.7.1.1.5 Détection et gestion des doublons

- Les doublons sont identifiés à partir d’une clé composite construite à partir de plusieurs attributs pertinents (par exemple : INE, entité concernée, période), permettant de représenter de manière unique un enregistrement métier.
- La composition de cette clé peut varier selon le type de donnée traité (étudiant, souhait, accord, etc.), afin de s’adapter aux spécificités de chaque entité.
- Chaque enregistrement est comparé aux données existantes afin de détecter les occurrences identiques.
- En cas de doublon exact, l’enregistrement est ignoré afin d’éviter la duplication des données.
- Les cas détectés sont tracés dans le rapport d’import pour assurer la traçabilité.

4.7.1.1.6 Validation métier

- Vérification de la cohérence avec les données Pégase (existence de l’étudiant via INE).
- Contrôle des règles métier (nombre maximum de vœux, compatibilité des destinations).

4.7.1.1.7 Traçabilité et gestion des erreurs

- Journalisation des erreurs dans un rapport d'import structuré.
- Stockage des données rejetées avec leur motif (audit et reprise).
- Attribution d'un identifiant de traitement pour chaque batch.

4.7.1.2 Architecture du pipeline de nettoyage

Le pipeline de nettoyage est conçu comme un module autonome intégré dans le flux global. Il fonctionne par batch afin d'assurer l'isolation des traitements et la robustesse du système.

- **Entrée** : données JSON issues du pipeline global.
- **Parsing** : transformation en objets manipulables.
- **Nettoyage** : standardisation et normalisation.
- **Validation** : application des règles métier.
- **Sortie** :
 - Données valides → persistance.
 - Données invalides → rapport d'erreurs.

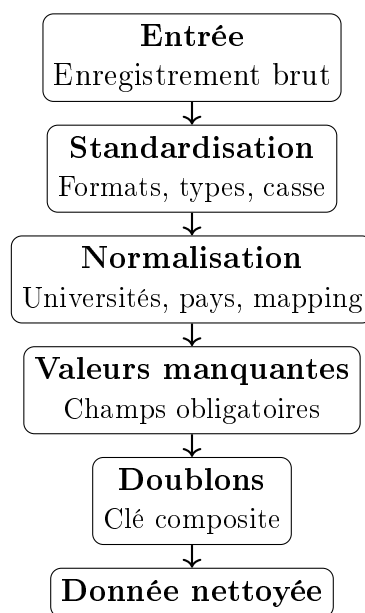


FIGURE 4.5 – Pipeline détaillé de l'étape de nettoyage des données

Cette architecture garantit :

- une **traçabilité complète** grâce au stockage JSONB,
- une **robustesse** via le traitement isolé de chaque enregistrement,
- une **reprise sur erreur** par batch,
- une **maintenabilité** grâce à la modularité du pipeline.

4.7.1.3 Technologies et outils utilisés

- **Python** : traitement et transformation des données.
- **Pandas** : manipulation et nettoyage des datasets.
- **Regex** : validation et transformation des formats.
- **PostgreSQL (JSONB)** : stockage temporaire des données brutes.

Ce pipeline garantit que seules des données fiables, cohérentes et conformes aux règles métier sont intégrées dans le système, assurant ainsi la robustesse globale du processus.

4.8 Gestion des risques

Les principaux risques identifiés lors de la conception du système, ainsi que les stratégies de mitigation retenues, sont présentés dans le tableau suivant.

TABLE 4.10 – Matrice des risques

Risque	Prob.	Impact	Mitigation
INE absent dans une source externe (Eudonet, MoveON)	Haute	Élevé	Réconciliation par matching nom / prénom / date de naissance. Cas ambigus remontés pour validation manuelle par le service RI.
API externe indisponible pendant une campagne	Moyenne	Élevé	Import Excel de secours documenté pour chaque source. Cache local des données de la dernière synchronisation réussie.
Dérive du périmètre en cours de PFE	Haute	Moyen	Priorisation stricte MVP / V2. Les fonctionnalités V2 (chatbot, recommandation) restent hors engagement initial et ne sont activées que dans une réserve conditionnelle, après validation du MVP et selon la capacité disponible.
Résistance au changement du service RI	Moyenne	Moyen	Implication du service RI dans les tests d'acceptation (UAT). Interface proche des pratiques Excel actuelles.

Risque	Prob.	Impact	Mitigation
Fuite de données personnelles	Faible	Très élevé	Chiffrement des données sensibles (pgcrypto), cloisonnement strict par rôle (RBAC), audit log de toutes les actions.
Démarrage à froid du moteur de recommandation	Haute	Faible	Les premières campagnes utiliseront des heuristiques simples (GPA + département). Le modèle s'affine progressivement à mesure que l'historique s'accumule (2-3 ans de données suffisent).
Performance de l'algorithme d'affectation sous contraintes multiples	Faible	Moyen	Gale-Shapley est en $O(n^2)$: parfaitement adapté à quelques centaines d'étudiants. Un test de charge Locust sera réalisé avant la mise en production pour vérifier les temps de réponse.

Conclusion

La conception du système est désormais complète : le backlog structure le développement en cinq sprints MVP, l'architecture définit les responsabilités de chaque couche et container, le modèle de données formalise les entités métier, et les machines à états garantissent l'intégrité des cycles de vie. Le chapitre suivant présente la réalisation sprint par sprint.

Réalisation

Plan

Introduction	82
5.1 Sprint 1 — Socle technique et référentiels	82
5.1.1 Objectif du sprint	82
5.1.2 Sprint Backlog — Sprint 1	82
5.1.3 Conception détaillée	84
5.1.3.1 Diagramme de classes	84
5.1.4 Implémentation	85
5.1.5 Tests	85
5.2 Sprint 2 — Données et traçabilité	85
5.2.1 Objectif du sprint	85
5.2.2 Sprint Backlog — Sprint 2	86
5.2.3 Conception détaillée	88
5.2.3.1 Diagramme de classes	88
5.2.4 Implémentation	89
5.2.5 Tests	89

Introduction

Ce chapitre couvre la phase de réalisation du projet, organisée en sprints successifs. Pour chaque sprint, le backlog, la conception détaillée, l'implémentation et les tests sont présentés conformément à la définition of done.

5.1 Sprint 1 — Socle technique et référentiels

Cette section présente la réalisation du Sprint 1, dédié à la mise en place du socle technique et des référentiels. Elle couvre successivement l'objectif du sprint, le backlog, la conception UML, l'implémentation et les tests.

5.1.1 Objectif du sprint

Le Sprint 1 a pour objectif d'implémenter les fondations irréversibles du système : référentiels stables (`Country`, `Department`), universités partenaires (`PartnerUniversity`), année universitaire avec sa machine à états à 6 phases (`AcademicYear`), accords et quotas (`Agreement`, `AgreementQuota`, `DepartmentQuota`) et stockage brut des imports (`RawImport`). Ces composants constituent la colonne vertébrale du système — toute fonctionnalité des sprints suivants en dépend directement.

Définition of Done. À l'issue de ce sprint, le système doit être exécutable via `docker compose up`, toutes les migrations Django doivent s'appliquer sans erreur, les tests unitaires doivent passer avec une couverture supérieure à 80 %, et le pipeline CI/CD doit être opérationnel sur GitHub Actions.

5.1.2 Sprint Backlog — Sprint 1

#	Tâche	Description	Critères d'acceptation	Prior.	SP
US-01 — Référentiels de base — <i>En tant qu'administrateur RI, je veux disposer d'un référentiel stable des pays, départements et universités partenaires afin que toutes les entités du système s'appuient sur des données de référence cohérentes.</i>					
01	Définir <code>Country</code> et <code>Department</code>	Créer les deux modèles de référentiel avec <code>iso2</code> unique, région CTI, <code>clean()</code> et <code>last_sync_moveon</code> .	Les modèles migrent sans erreur.	Haute	2
02	Définir <code>PartnerUniversity</code>	Créer le modèle avec <code>moveon_id</code> comme clé de jointure <code>MoveON</code> .	Le modèle migre sans erreur.	Haute	1

#	Tâche	Description	Critères d'acceptation	Prior.	SP
03	Page d'administration des référentiels	Créer une interface Next.js permettant à l'administrateur RI de consulter et filtrer les pays, départements et universités partenaires enregistrés.	La liste s'affiche, le filtre par nom fonctionne, les données sont chargées depuis l'API.	Haute	2
US-02 — Année universitaire et cycle de vie — <i>En tant qu'administrateur RI, je veux que le système gère le cycle de vie de l'année universitaire via une machine à états afin de contrôler chaque phase de la campagne de manière sécurisée.</i>					
04	Définir AcademicYear avec FSM	Définir le modèle AcademicYear en utilisant une machine à états finis (FSM), implémentée avec <code>django-fsm</code> , comprenant 6 états correspondant aux différentes phases de la campagne. Mettre en place des transitions sécurisées entre ces états, ainsi qu'une méthode <code>clean()</code> permettant de garantir la cohérence des dates.	Les transitions valides fonctionnent. Les invalides lèvent <code>TransitionNotAllowed</code> . <code>start_date > end_date</code> lève <code>ValidationError</code> .	Haute	2
05	Composant de gestion de l'année universitaire	Créer une interface Next.js affichant l'état courant de l'année universitaire avec les boutons de transition disponibles (selon la FSM) et les dates clés configurables par l'administrateur.	L'état affiché reflète la FSM. Les transitions non autorisées sont masquées. Les dates se sauvegardent via l'API.	Haute	2
US-03 — Accords et quotas — modèle de données — <i>En tant qu'administrateur RI, je veux disposer d'un modèle de données complet pour les accords et leurs quotas afin d'avoir une base fiable pour l'algorithme d'affectation.</i>					
06	Définir Agreement	Créer le modèle avec <code>moveon_id</code> , <code>code_moveon</code> unique, <code>agreement_type</code> , <code>valid_from/until</code> , <code>is_active</code> et <code>last_sync_moveon</code> .	Le modèle migre sans erreur. <code>is_valid_for_year()</code> retourne <code>False</code> pour un accord expiré ou inactif.	Haute	2
07	Définir AgreementQuota et DepartmentQuota	Créer les modèles de quotas avec hiérarchie INP → N7 estimé → N7 ajusté.	<code>get_effective_quota()</code> retourne l'ajusté si renseigné, sinon l'estimé. <code>is_eligible()</code> est fonctionnel.	Haute	2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
08	Définir RawImport	Créer le modèle de stockage brut JSON avec <code>source</code> , <code>processed</code> , <code>error_message</code> et rattachement à <code>AcademicYear</code> .	Le modèle migre sans erreur. Les 4 sources sont disponibles. <code>processed=False</code> par défaut.	Haute	1
09	Page de consultation des accords et quotas	Créer une interface Next.js listant les accords par université partenaire, avec affichage des quotas INP, estimé N7 et ajusté N7 par département, et un indicateur visuel de validité de l'accord.	Les accords s'affichent avec leurs quotas effectifs. Les accords expirés ou inactifs sont signalés visuellement. L'ajustement manuel du quota N7 est possible depuis l'interface.	Haute	3

5.1.3 Conception détaillée

Cette section présente la conception UML détaillée des entités métier du Sprint 1.

5.1.3.1 Diagramme de classes

Le diagramme de la figure 5.1 modélise le domaine de gestion des accords de mobilité internationale de l'ENSEEIH. L'entité centrale est **Agreement**, qui représente un accord entre l'ENSEEIH et une université partenaire (**PartnerUniversity**), qualifiée par un cadre (**Category**) et des niveaux d'études (**Level**).

La structure des quotas est organisée sur deux niveaux : **AgreementYear** matérialise la déclinaison annuelle d'un accord avec le quota N7 global, et **AgreementYearDepartment** distribue ce quota par département. **AgreementDepartment** liste les départements autorisés par accord, indépendamment de l'année, et constitue la contrainte référentielle qui interdit d'allouer un quota à un département non autorisé.

AcademicYear pilote le cycle de vie de la campagne via une machine à états à six phases. Toutes les classes portent les champs d'audit `created_at`, `updated_at`, `created_by` et `updated_by`.

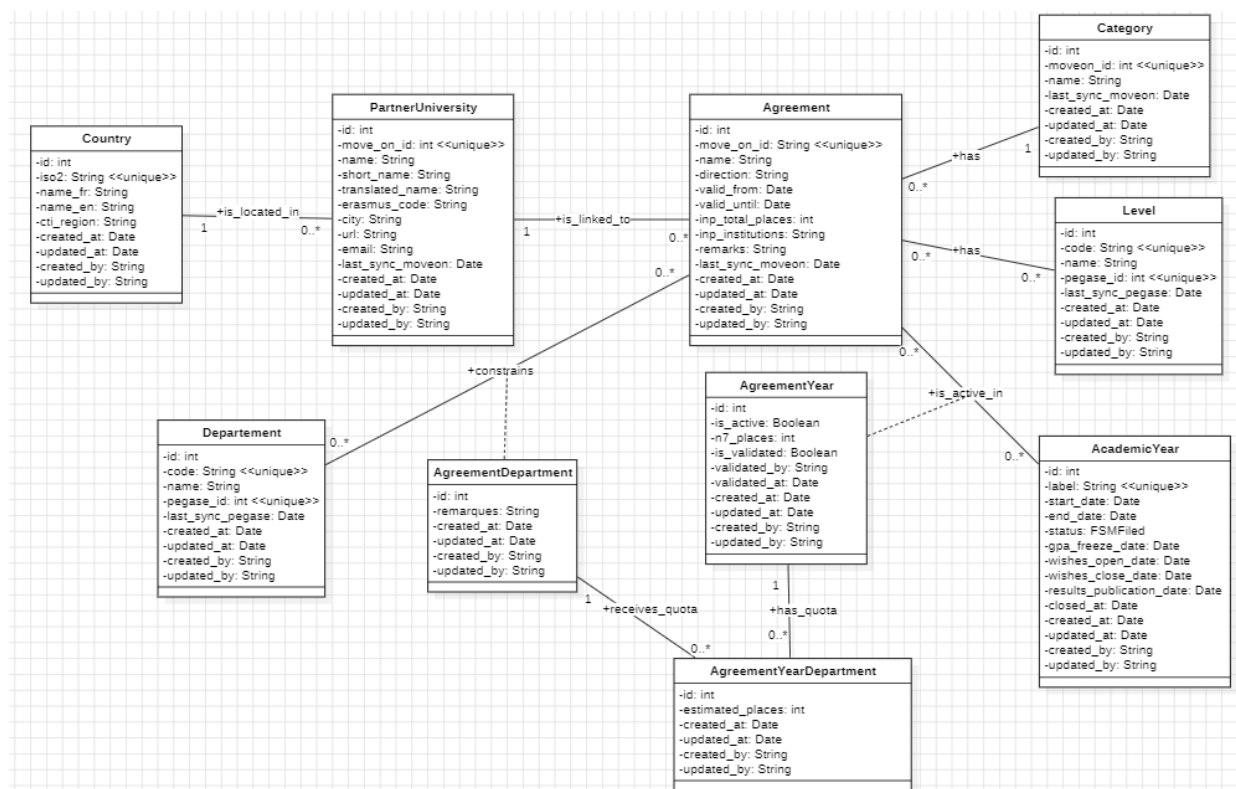


FIGURE 5.1 – Diagramme de classes — Sprint 1

5.1.4 Implémentation

5.1.5 Tests

5.2 Sprint 2 — Données et traçabilité

Cette section présente la réalisation du Sprint 2, consacré à la gestion des étudiants, de leurs inscriptions annuelles et de leurs vœux de mobilité. Elle couvre successivement l'objectif du sprint, le backlog, la conception UML, l'implémentation et les tests.

5.2.1 Objectif du sprint

Le Sprint 2 a pour objectif de rendre le système opérationnel pour la collecte et la centralisation des données. Il couvre quatre user stories : la traçabilité des actions via l'audit log (US-04), l'import et la synchronisation des données académiques étudiants depuis Pégase (US-05), l'import des dossiers de candidature et vœux depuis MoveON (US-06), et la génération des rapports d'import (US-07). Ces quatre user stories sont le prérequis indispensable au Sprint 3 : sans données étudiants fiables issues de Pégase, le gel du GPA et l'algorithme Gale-Shapley ne peuvent pas s'exécuter.

Définition of Done. À l'issue de ce sprint, les imports Pégase et MoveON s'exécutent sans erreur, les données sont persistées en base et rattachées à l'année universitaire cou-

rante, un rapport d'import est généré pour chaque synchronisation, et toute action d'un administrateur RI est tracée dans l'audit log avec l'auteur et l'horodatage.

5.2.2 Sprint Backlog — Sprint 2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
US-04 — Audit log et traçabilité — <i>En tant qu'administrateur RI, je veux que chaque action importante (import, transition FSM, modification, validation) soit enregistrée avec l'auteur et l'horodatage, afin de garantir la traçabilité complète des décisions.</i>					
01	Définir AuditLog	Créer le modèle avec <code>user</code> , <code>action</code> , <code>entity_type</code> , <code>entity_id</code> , <code>payload</code> JSONB et <code>timestamp</code> . Connecter via signal Django <code>post_save</code> sur les modèles critiques.	Le modèle migre sans erreur. Chaque transition FSM et chaque import créent une entrée AuditLog avec l'auteur.	Haute	2
02	Endpoint consultation audit	Exposer GET <code>/api/audit-logs</code> avec filtres par date, auteur et type d'entité via Django Ninja.	Les 20 dernières entrées s'affichent en moins de 200 ms. Les filtres date/auteur/type fonctionnent correctement.	Haute	1
US-05 — Données étudiants — <i>En tant qu'administrateur RI, je veux importer et synchroniser les données académiques des étudiants (INE, GPA, département, niveau) depuis Pégase, afin d'alimenter l'algorithme d'affectation avec des données étudiants fiables et à jour.</i>					
03	Créer adaptateur <code>pegase.py</code>	Implémenter l'extraction des données académiques (INE, GPA, département, niveau) depuis l'API Pégase. Stocker la réponse brute en JSONB dans <code>RawImport</code> .	La réponse brute est stockée avec <code>source="pegase"</code> . Retry exponentiel (3 tentatives) en cas d'échec.	Haute	2
04	Définir <code>Student</code> et <code>AnnualEnrollment</code>	Créer le modèle <code>Student</code> (INE comme clé unique, nom, prénom, email institutionnel) et <code>AnnualEnrollment</code> (inscription annuelle avec GPA, département, niveau, rattaché à <code>AcademicYear</code>).	Les modèles migrent sans erreur. Un étudiant peut avoir plusieurs inscriptions sur des années différentes. INE est unique sur <code>Student</code> .	Haute	2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
05	Pipeline ETL Pégase	Transformer les données brutes en entités <code>Student</code> et <code>AnnualEnrollment</code> . Upsert sur INE. Le GPA est stocké tel quel (gel ultérieur en Sprint 3).	Les étudiants sont créés ou mis à jour. Les doublons sont ignorés. Un rapport d'import est généré avec les anomalies détectées.	Haute	3
06	Interface consultation étudiants	Créer une page Next.js listant les étudiants importés avec leur GPA, département et année d'inscription courante.	La liste s'affiche avec filtre par département et niveau. Les données proviennent de l'API.	Haute	1
US-06 — Vœux étudiants — <i>En tant qu'administrateur RI, je veux importer les dossiers de candidature et les vœux ordonnés des étudiants depuis MoveON, et les consulter par étudiant, afin de les intégrer dans le système d'affectation.</i>					
07	Définir <code>ApplicationFile</code> et <code>Wish</code>	Créer le modèle <code>ApplicationFile</code> (dossier de candidature, FSM à 4 états : <code>open</code> → <code>submitted</code> → <code>processing</code> → <code>closed</code>) et <code>Wish</code> (rang, université cible, département préféré, rattaché à <code>AcademicYear</code>).	Les modèles migrent sans erreur. Les transitions FSM invalides lèvent <code>TransitionNotAllowed</code> . Un <code>Wish</code> ne peut exister sans <code>ApplicationFile</code> .	Haute	2
08	Pipeline ETL vœux MoveON	Importer les vœux depuis MoveON, croiser les INE avec la table <code>Student</code> , rejeter les entrées sans correspondance INE.	Les vœux sont persistés avec rang et université cible. Les INE introuvables sont consignés dans le rapport avec motif <code>INE_NOT_FOUND</code> .	Haute	2
09	Interface consultation vœux	Créer une page Next.js listant les dossiers de candidature par étudiant avec le détail des vœux et l'état FSM de chaque dossier.	La liste s'affiche avec filtre par étudiant et état. Les données sont chargées depuis l'API.	Haute	1
US-07 — Rapport d'import — <i>En tant qu'administrateur RI, je veux consulter un rapport détaillé après chaque import (succès, erreurs, doublons) avec la raison précise de chaque rejet, afin de corriger les anomalies rapidement.</i>					

#	Tâche	Description	Critères d'acceptation	Prior.	SP
10	Définir ImportReport	Créer le modèle avec <code>source</code> , <code>total</code> , <code>success_count</code> , <code>error_count</code> , <code>duplicate_count</code> et <code>errors JSONB</code> (motif et identifiant par entrée rejetée), rattaché à <code>RawImport</code> .	Chaque pipeline ETL crée automatiquement un rapport. Les erreurs contiennent motif et numéro de ligne ou identifiant source.	Haute	1
11	Page rapport d'import	Créer une interface Next.js listant les rapports par date et source avec les compteurs synthétiques et le détail des erreurs accessible par clic.	Les rapports s'affichent triés par date. Le détail des erreurs est accessible. Le filtre par source fonctionne.	Haute	2

5.2.3 Conception détaillée

Cette section présente la conception UML détaillée des entités métier du Sprint 2.

5.2.3.1 Diagramme de classes

Le diagramme de la figure 5.2 modélise la gestion des étudiants et de leurs vœux de mobilité au sein du système de mobilité de l'ENSEEIH.

AnnualEnrollment est la classe d'association centrale du sprint : elle matérialise l'inscription d'un étudiant (**Student**) pour une année universitaire (**AcademicYear**) donnée, et porte le département et le niveau d'études. Le parcours est renseigné uniquement lorsque l'étudiant a intégré une filière spécialisée ; il reste absent pour les niveaux en tronc commun (première année). **StudentWish** matérialise à son tour un vœu de mobilité émis dans le cadre d'une inscription annuelle : chaque inscription peut générer au plus trois vœux, chacun pointant vers un accord différent et portant un rang et un statut.

Une contrainte d'intégrité garantit que le parcours suivi par l'étudiant appartient au même département que son inscription annuelle, exprimée par la contrainte :

`{parcours.department = department}` annotée sur l'association.

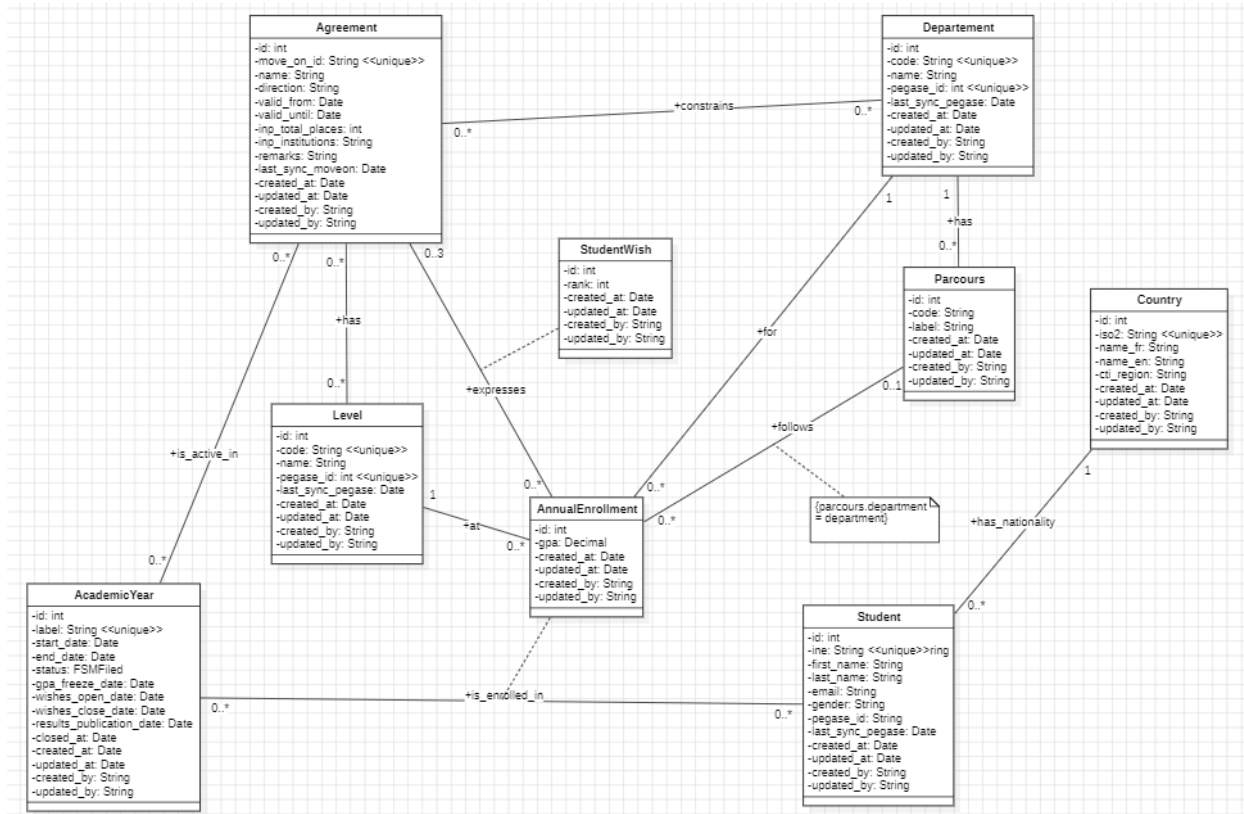


FIGURE 5.2 – Diagramme de classes — Sprint 2

5.2.4 Implémentation

5.2.5 Tests

Conclusion générale

Bibliographie

- [1] *L'ENSEEIH*T – *Présentation*. <https://www.enseeiht.fr/fr/l-enseeiht.html>. Consulté le 2026-03-18.
- [2] *International* – *Toulouse INP-ENSEEIH*T. <https://www.enseeiht.fr/fr/international.html>. Consulté le 2026-03-18.
- [3] *Mobilité sortante* – *Toulouse INP-ENSEEIH*T. <https://www.enseeiht.fr/fr/international/mobilite-sortante.html>. Consulté le 2026-03-18.
- [4] *Mobilité entrante* – *Toulouse INP-ENSEEIH*T. <https://www.enseeiht.fr/fr/international/mobilite-entrante.html>. Consulté le 2026-03-18.
- [5] *Programmes courts* – *Toulouse INP-ENSEEIH*T. <https://www.enseeiht.fr/fr/international/programmes-courts.html>. Consulté le 2026-03-18.
- [6] *MoveON* – *QS Solutions*. <https://www.qs.com/solutions/moveon>. Consulté le 2026-03-09.
- [7] *Définition CRM* – *Eudonet*. <https://www.eudonet.com/fr/ressources/blog/definition-crm/>. Consulté le 2026-03-09.
- [8] Mehdi ELAHI et al. « Developing and Evaluating a University Recommender System ». In : *Frontiers in Artificial Intelligence* (2022). Consulté le 2026-03-09. URL : <https://pmc.ncbi.nlm.nih.gov/articles/PMC8848746/>.
- [9] *React* – *The library for web and native user interfaces*. <https://react.dev/>. Consulté le 2026-03-17.
- [10] *Angular* – *The web development framework*. <https://angular.dev/>. Consulté le 2026-03-17.
- [11] *Next.js* – *The React Framework for the Web*. <https://nextjs.org/>. Consulté le 2026-03-17.
- [12] *Node.js* – *Run JavaScript Everywhere*. <https://nodejs.org/>. Consulté le 2026-03-18.
- [13] *FastAPI* – *Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com/>. Consulté le 2026-03-18.
- [14] *Django* – *The web framework for perfectionists with deadlines*. <https://www.djangoproject.com/>. Consulté le 2026-03-18.
- [15] *Django REST Framework* – *Web APIs for Django*. <https://www.django-rest-framework.org/>. Consulté le 2026-03-18.
- [16] *Django Ninja* – *Fast Django REST Framework*. <https://django-ninja.dev/>. Consulté le 2026-03-18.

- [17] *MySQL – The world’s most popular open source database.* <https://www.mysql.com/>. Consulté le 2026-03-17.
- [18] *MongoDB – The developer data platform.* <https://www.mongodb.com/>. Consulté le 2026-03-17.
- [19] *PostgreSQL – The world’s most advanced open source relational database.* <https://www.postgresql.org/>. Consulté le 2026-03-17.
- [20] *Celery – Distributed Task Queue.* <https://docs.celeryq.dev/>. Consulté le 2026-03-19.
- [21] *Redis – The Real-time Data Platform.* <https://redis.io/docs/>. Consulté le 2026-03-19.
- [22] *Django Q2 – A multiprocessing task queue for Django.* <https://django-q2.readthedocs.io/>. Consulté le 2026-03-19.
- [23] *MinIO – High Performance Object Storage.* <https://min.io/docs/>. Consulté le 2026-03-24.
- [24] *Nginx – High Performance Load Balancer, Web Server, & Reverse Proxy.* <https://nginx.org/en/docs/>. Consulté le 2026-03-27.
- [25] *Traefik – The Cloud Native Application Proxy.* <https://doc.traefik.io/traefik/>. Consulté le 2026-03-27.
- [26] *Scikit-learn – Machine Learning in Python.* <https://scikit-learn.org/stable/>. Consulté le 2026-03-24.
- [27] *Mistral AI – La plateforme pour les modèles de langage.* <https://docs.mistral.ai/>. Consulté le 2026-03-30.
- [28] *LangChain – Build context-aware reasoning applications.* <https://python.langchain.com/docs/>. Consulté le 2026-03-30.
- [29] *LlamaIndex – Python Framework Documentation.* <https://developers.llamaindex.ai/python/framework/>. Consulté le 2026-03-30.
- [30] *Haystack – The open-source AI framework.* <https://docs.haystack.deepset.ai/>. Consulté le 2026-03-30.